🚀 Background / Description

Box contents transcription

University of Melbourne Archives (UMA) was founded in 1960, long before electronic databases existed. Early inventories of the archive sometimes took the form of typed lists of documents, but often archivists would write by hand notes about the contents of the boxes directly on the face of the box. We currently have thousands of boxes with these handwritten annotations that require transcription. Manually transcribing the detail of the contents of each box is time consuming and laborious. We propose a computer vision application that would take an image, segment it into boxes, transcribe the handwritten contents and any labels attached to the box, and output a csv or xlsx inventory. The application would need some flexibility with network coverage as the archive repository does not have complete wireless coverage.

Team Social Details

Name - Email Address - Github Username

Ulrich Fuhring - [redacted] - Ulrich-Fuhring

Amy Mauseth - [redacted] - Siri-chan

Liem Edwardes - [redacted] - ledwardes

Julius Tee - [redacted] - jtee27

Zijie Xu - [redacted] - 91woshinailong

🧩 Individual Roles & Skillsets

This document outlines the initial proposition for each team members' role.

Roles will be flexible but we wanted group members with domain-focused knowledge for each major component. This is the current allocation for general roles:

Ulrich (Product Owner)

- Box detection
- Sticker detection

Amy

- Frontend command line interface
- Meta-Architecture
- System Integration

Liem

- Project documentation
- Design Lead
- CSV parsing from OCR
- Testing on Python modules

Julius (Scrum Master)

- OCR text-extraction

Xu

- Frontend command line interface

Role Descriptions

Here are formal definitions of each team member's responsibilities.

Amy - Architecture Lead and Secretary

As Architecture Lead

Responsible for the project's high-level structure, and inter-operation between other team members' modules.

In particular, this involves:

- CI/CD
- Gradle Project Management
- The `Switchboard`
- Logging
- Python Interoperation

As Secretary

Responsible for organising meeting agendas alongside **Julius** and taking minutes.

Julius - OCR Lead and Scrum Master

As OCR Lead

Responsible for implementation, maintainance and improvement of an OCR Machine Learning engine.

As Scrum Master

Responsible for keeping meetings on-task, managing sprints and coordinating agendas alongside **Amy**.

Liem - Documentation, Design, and GUI Lead

As Documentation Lead

Responsible for the organisation and quality of documentation.

As GUI and Design Lead

Responsible for designing and development a GUI for 'Box Detection' result review and a GUI for 'OCR' result review.

Ulrich - Box Detection Lead and Product Owner

As Box Detection Lead

Responsible for implementation, maintenance, and improvement of an Object-Detection Machine Learning Engine.

As Product Owner

Responsible for Communication with Client and Prioritisation of Epics during Sprint Planning

Xu - CLI Lead

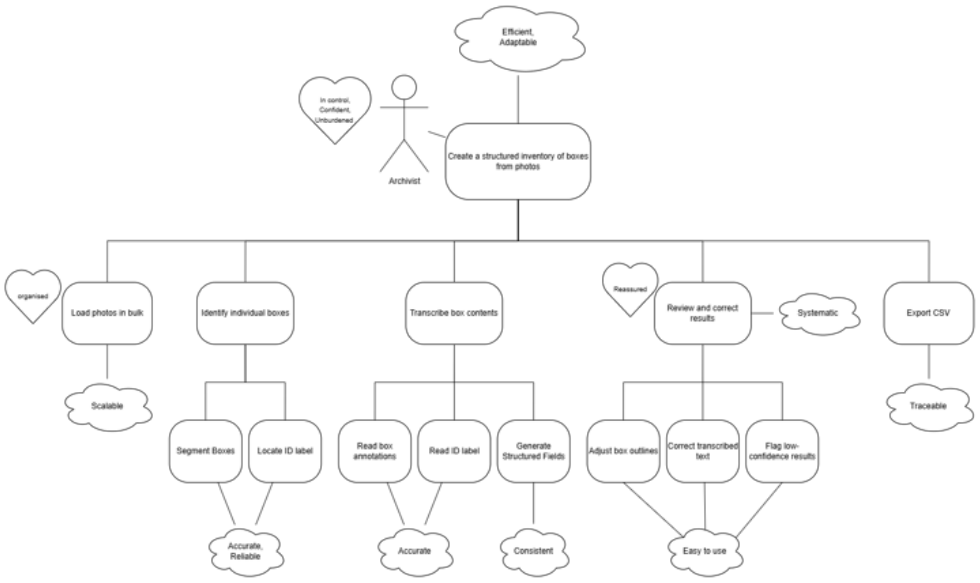
As CLI Lead

Responsible for designing, implementing and maintaining a high-quality command-line interface.

Do/Be/Feel/Who

Who	Do	Be	Feel
Archivists who work with and support the University of Melbourne Archives	Digitise the University of Melbourne Archives from photos of physical records Identify individual archive records Extract text and divide into structured, sortable fields of data Review and validate digitised archive data	Flexible Consistent Efficient Scalable Systematic	In control Clear Robust

✨ Motivational Model



💻 User Stories

User Story Table

Number	Allocation (1-5)	Epic	User Story	Priority (Low, Medium, High)
1	4	Digitise the University of Melbourne Archives from photos of physical records	As an archivist, I **need** a usable interface that facilitates the usage of the software.	High
2	5	Identify individual archive records	As an archivist, I **need** to automatically identify and separate a shelf of boxes so that I don't have to manually take photos of each.	High
3	1	Identify individual archive records	As an archivist, I **want** to be able to use the archive's existing unique reference numbers to identify a box, so that I don't have to learn and implement a new reference system.	Medium
4	-	Identify individual archive records	As an archivist, I **don't want** the output to be indexed by the position of a box within an image because it doesn't help me locate the boxes, and it has no meaning without the input images.	-

5	2	Identify individual archive records	As an archivist, I **need** to be able to separate the ID stickers from the ‘miscellaneous data’ on the rest of the box, so that I can more easily identify which boxes the application refers to.	High
6	5	Extract text and divide into structured, sortable fields of data	As an archivist, I **need** to be able to extract text from an isolated archive record.	High
7	2	Extract text and divide into structured, sortable fields of data	As an archivist, I **want** any identified date-ranges to be labelled in our output to be included additionally to it’s appearance in the text, so that I can easily distinguish items of the same collection, where the text may otherwise be long and mostly identical.	Medium
8	2	Extract text and divide into structured, sortable fields of data	As a reviewer, I **might** want the ability to detect irrelevant or separable information.	Low

9	-	Extract text and divide into structured, sortable fields of data	As an reviewer, I **don't want** excessive metadata or categories to be generated from the box's text, because that creates a lot of unnecessary noise.	-
10	1	Extract text and divide into structured, sortable fields of data	As a data-scientist, I **want** a well-structured and easy-to-work-with output format so that I can integrate the application with new-and-existing scripts.	Medium
11	1	Identify individual archive records	As an archivist, I **want** an easy-to inspect and edit segmentation output format so that I can review and correct the output easily.	Medium
12	2	Review and validate digitised archive data	As a reviewer, I **want** to know which entries are least coherent, so that I can prioritise them for review.	Medium
13	5	Review and validate digitised archive data	As a reviewer, I **want** to be able to review the generated bounding boxes (or the resultant images), so that I can adjust any that don't align properly.	Medium
14	2	Review and validate digitised archive data	As a developer, I **might** want to design an interface, so that OCR output can be manually adjusted.	Low

15	3	Review and validate digitised archive data	As a reviewer, I **want** to be able to merge and easily deduplicate the results of multiple runs, so that I can easily segment my work.	Medium
16	3	Review and validate digitised archive data	As a reviewer, I **might** want an interface, so that OCR output can be manually adjusted.	Low
17	4	Review and validate digitised archive data	As an archivist, I **might** want the ability to aggregate multiple images to help with legibility.	Low
18	5	Use a functional application	As a user, I **want** a single cohesive application that can work from end-to-end, so that I'm not downloading or trying to fix several disconnected tools.	Medium
19	1	Digitise the University of Melbourne Archives from photos of physical records	As a data-scientist, I **might want** be able to tweak the Machine Learning components in a familiar language, so that I can adjust the model if it isn't performing as expected.	Low
20	3	Extract text and divide into structured, sortable fields of data	As a data-scientist, I **might want** to use a part of the software pipeline as a standalone tool, so that I can use it for other applications.	Low

21	3	Digitise the University of Melbourne Archives from photos of physical records	As a developer, I **need** the architectural workflow of the CLI commands defined and validated, so that I can implement the CLI consistently with the system architecture.	High
22	2	Review and validate digitised archive data	As a developer, I **need** to design an interface, so that segmentation can be visually adjusted.	High
23	2	Identify individual archive records	As an archivist, I **want** the system to skip OCR on boxes without any sticker, so that processing time isn't wasted on out-of-scope boxes.	Medium
24	5	Digitise the University of Melbourne Archives from photos of physical records	As a user, I **need** a program that implements all of the functionality it claims to provide, so that I can trust the program to successfully complete my work.	High

Sprint 1 Plan



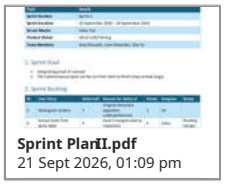
Sprint Plan I.pdf
06 Sept 2026, 03:26 pm

Sprint 1 Retrospective



Sprint Retrospe..I).pdf
10 Sept 2026, 05:26 pm

Sprint 2 Plan



Sprint PlanII.pdf
21 Sept 2026, 01:09 pm

🪄 Sprint 2 Retroactive

Field	Details
Sprint Number	Sprint 2
Date	24 September 2026
Present	Julius, Ulrich, Amy, Liem , Xu

Key successes

- Agile ceremonies were conducted smoothly.
- Significant progress has been made in the software's development
  - Code for OCR implementation and output is functioning and was merged into the Switchboard's branch.
  - GUI for box segmentation review is complete.
  - CLI implementation was integrated into the concrete Switchboard.
  - Box segmentation model achieved excellent results (i.e. accurately drew bounding boxes for the boxes and stickers).
- Group's progress and momentum was maintained despite having a few members getting sick.

Hurdle points

- Some Trello cards were too big (i.e. too many key goals to accomplish)
  - Example: User Story #24 (Concrete Interaction Implementation)
  - It has too many action items, but at the same time it was difficult to further divide it into smaller cards.
- There were difficulties in integrating Python code into Kotlin.
- No concrete test plans were made.

Actionable items

- Collaborate and finalise a concrete test plan
- Developers should finalise their portion of the project to **develop a functioning prototype**.

🚀 Sprint 3 Plan

Sprint Overview

Fields	Details
Sprint Number	Sprint 3
Sprint Duration	24 September 2026
Scrum Master	Julius
Product Owner	Ulrich
Team Members	Amy, Liem, Xu

Sprint Goals

- **Implement quality assurance (QA)** to improve and maintain the overall software quality.
- **Produce a rudimentary but functional prototype**.

Sprint Backlog

ID	User Story	Deferred?	Reason for deferral	Pts.	Assignee	Status
3	Unique Reference Number Reading	N	-	1	Julius	Complete
12	Coherent Entry Ranking	N	-	2	Amy	In progress
14	Design OCR Review Interface	N	-	2	Liem	Complete
15	Duplication Detection	N	-	3	Amy	In progress
16	Implement OCR Review Interface	N	-	4	Liem	In progress

19	Allow the Models to be Adjustable	N	-	1	Ulrich	Complete
23	Filter Out Boxes Without Stickers	N	-	2	Amy	In progress
24	Concrete Interaction Implementations	Y	Large user story (contains many key goals)	5	Amy	In progress

Total commited points: **19**

Definition of Done (DoD)

Checklist that must be fulfilled before a task/story is considered complete:

- Code written and peer-reviewed
- Unit tests written and passing
- Feature tested against acceptance criteria
- Documentation updated
- Product Owner signed off
- Code committeed to repository

Sprint Ceremonies Schedule

Ceremony	Date / Time	Purpose
Sprint planning	24 September, 5 pm - 6 pm	Define goal, product and sprint backlogs
Weekly official stand-ups	Every Monday, 2 pm - 3 pm Every Thursday, 4 pm - 6 pm	Sync progress
Backlog refinement	At meetings	Groom user stories

Sprint review	Before 8 October (contingent on client's availability)	Provide progress update and direction to client
Sprint retrospective	8 October	Reflect on improvements for the upcoming sprints

🪄 Sprint 3 Retroactive

Field	Details
Sprint Number	Sprint 3
Date	8 October 2026
Present	Julius, Ulrich, Amy, Liem , Xu

Key successes

- Recent client meetings have been fruitful and satisfactory, especially the most recent one.
- Organised remote meetings without issues (during the mid-semester break)
- Group members were flexible around changing working conditions of the mid-semester break
- Coped well with illness of team members
- Project is finished enough that the code is usable

Hurdle points

- Card #24 is still absolutely monolithic despite fixing the others
- Group members were pidgeon-holed in Python code
- Problems with the Python code should have been caught earlier
- Time management could have been better, but we also had to cope with other committments.

Actionable items

- Focus on documentation
  - Test plan
  - Centraise architecture properly
  - Finalise user-facing documents
  - Prepare the documentation of decisions

🚀 Sprint 4 Plan

Sprint Overview

Fields	Details
Sprint Number	Sprint 4
Sprint Duration	8 October 2026 - 26 October 2026
Scrum Master	Julius
Product Owner	Ulrich
Team Members	Amy, Liem, Xu

Sprint Goals

- Accomplish all the remaining user stories that are carried over from the previous sprint.
- Finalise all the necessary parts of the Week 12 Progress Report.
- Prepare for the oral presentation.

Sprint Backlog

ID	User Story	Deferred?	Reason for deferral	Pts.	Assignee	Status
12	Coherent Entry Ranking	Y	Ran into issues with building the app.	2	Ulrich	In progress
15	Duplication Detection	Y	Prioritised other more important aspects of the software architecture	3	Amy	In progress
23	Filter out boxes without stickers	Y	Ran into issues with building the app.	2	Ulrich	In progress

24	Concrete Interaction Implementations	Y	Large user story (contains many key goals)	5	Amy	In progress

Total commited points: **12**

Definition of Done (DoD)

Checklist that must be fulfilled before a task/story is considered complete:

- Code written and peer-reviewed
- Unit tests written and passing
- Feature tested against acceptance criteria
- Documentation updated
- Product Owner signed off
- Code committeed to repository

Sprint Ceremonies Schedule

Ceremony	Date / Time	Purpose
Sprint planning	8 October, 5 pm - 6 pm	Define goal, product and sprint backlogs
Weekly official stand-ups	Every Monday, 1 pm - 2 pm Every Thursday, 4 pm - 6 pm	Sync progress
Backlog refinement	At meetings	Groom user stories
Sprint review	Before 8 October (contingent on client's availability)	Provide progress update and direction to client
Sprint retrospective	26 October	Reflect on improvements for the upcoming sprints

🪄 Sprint 4 Retroactive

Field	Details
Sprint Number	Sprint 4 (Final)
Date	26 October 2026
Present	Julius, Ulrich, Amy, Liem Xu (Apologised)

Key successes

- Code successfully met the minimum requirements by the Oral Presentation (Thursday, 22 October).
- Surpassed previous sprints on keeping up with the documentation.
- Communication amongst team members has improved.
- Completed all task boards (i.e. implemented and tested all functional and non-functional requirements in Trello).

Hurdle points

- Started too late with the preparation for the Oral Presentation.
  - In hindsight, if we have done more preparation, we could have avoided the technical issue that occured during the Oral Presentation.

Actionable items

N/A

Client Meetings

First Client Meeting (24/08)

Please see the " Week 5 Monday Client Meeting " section for documentation on our first team-organised meeting with the client.

Second Client Meeting (3/09)

Date: Thursday, 3 September 2026

Location: University of Melbourne Archives (UMA)

Attendees:

- Ulrich - Product Owner
- Julius - Scrum Master
- Lachlan Glanville - Client (representing the UoM Archives Team)

Context

An in person site-visit was organised with Lachlan (see Client Emails) to give project updates, clarify user requirements and collect further data for training our segmentation and text extraction models.

User Requirement Updates

Before taking photos for digitisation we approached the client with clarifying questions for our User Requirements, specifically focusing on the Epics relating to *Identifying individual Archive Records* and *Text Extraction*.

Identify individual Archive records

Box Detection

- Boxes without stickers **do not require** OCR or segmentation
  - These have not been processed yet and do not require digitisation
- The Archivists have already been using Gemini for some of their own transcription work
  - Lachlan commented on the difficulty of getting accurate transcriptions for the hand-writing as a known issue
- **Books are out of scope** for detection
- White boxes are **in scope but not a priority**



Label Variation Example

Label Detection

- Two label colours were verbally confirmed as in scope:
  - **Pink** - in scope
  - **White** - in scope, most common instance



Standard (Labelled) Brown Boxes and Unlabelled Books

Text Extraction

- Struck through text is out of scope and should be **ignored**
- The client requested a tool that shows the cropped box photo alongside the extracted text for human review

Date detection

- The earliest and latest detected dates from text extraction may be treated as the “start” and “end” dates without further clarification - these may be fixed in review
- Boxes with only one detected date may be treated as having a start date with no end date.
- The unit number written on the box are not canonical unless written on the sticker

General Q&A

- **Deliverable format:** A `.jar` is acceptable - the archive uses Windows machines, so both `.jar` and `.exe` would work.
- **Annotated training examples:** The client confirmed these can be provided.

Outcomes

From these new specifications and discussion with the client, the following decisions were taken (corresponding to added User Stories 23-29).

1. The OCR pipeline will **skip** boxes that have no sticker at all.
2. Detection scope excludes books; white boxes are included but deprioritised relative to stickered boxes.
3. Only pink and white labels are handled in the current scope.
4. Struck-through text is filtered out of OCR output before it reaches review.
5. Date ranges are **flagged, not resolved** — final call sits with the human reviewer.
6. A single detected date is interpreted as a start date only.
7. Box unit numbers are **not** used for identification unless also present on a sticker.
8. Application deliverable can be packaged as a `.jar`.

Third Client Meeting (24/08)

Please see the “ Week 5 Monday Client Meeting ” section for documentation on our third team-organised meeting with the client.

Forth Client Meeting (5/10)

Please see the “ Week 10 Monday Client Meeting ” section for documentation on our fourth team-organised meeting with the client.

(content removed)

🪄 Style Coding Standard

Code Formatting

Python is to be formatted with black (still have to confirm)

Kotlin is to be formatted with `ktfmt`. This can be run automatically with `./gradlew ktfmtFormat`

Naming Conventions

Element	Naming Case
Package	lowercase
Class / Interface	PascalCase
Variable	camelCase
Enum	PascalCase
Function	camelCase
Constant	UPPER_SNAKE_CASE
Test Function	lowercase with spaces This is done using backticks: `fun `test name goes here`()`

Code Documentation

Leave comments that explain functional blocks of code and decisions

Avoid comments that repeat the code directly (solutions should not be complex enough to warrant this)

🪄 Version Control Standards

Commit Standards

All commits must be written using Conventional Commits.

Full specification with examples and FAQ is provided below:

⭕ Conventional Commits

Branching Standards

Branch names should be a one-word description of the feature or fix.

Most branches should branch directly off of main and should be kept up to date with main unless they are blocked by another PR.

Pull Requests

Pull requests should be titled using a one-sentence description of the changes.

The Pull Request should be an isolated set of changes branching directly from `main`. If the PR is blocked by any others it should reference them by number (Github uses the format #<number>, which should auto-complete while in Github), or if it is a draft and has things still to complete, the description should contain a checklist.

Once a Pull Request is complete (not blocked and not marked as a draft), somebody should review it. This involves looking over the code using the 'Submit Review' button on the 'Files Changed' page and providing some criticism, the PR author should then go back and address said criticism. This should be a back and forth until everything is resolved.

Pull Requests are automatically checked using `./gradlew check` to ensure that all tests pass, and that all Kotlin code adheres to the 🪄 Style Coding Standard.
Amy will also include some tools to allow for the same level of automation for Python code in the future.

Once the reviewer is happy and the automated tests pass, they can mark the PR as approved, and merge it.

🚧 [Deprecated] Test Plan

Types of Test

Our Tests are broken down into 4 categories:

- Unit Tests
- Semi-Manual Tests
- Model Tests
- Integration Tests

These categories describe the different test processes that we use to handle different types of test, and how we implement them.

Unit Tests

Unit Tests are the most important type in our suite. They are automated tests, run through Gradle's testing framework, and run automatically through Github Actions.

Unit Tests should be defined in the `test` SourceSet of the module in which the 'Unit' is implemented (eg. `lib/src/test/`).

Kotlin

Unit Tests in Kotlin should be written using the `kotlin.test` library. The library API is heavily inspired by JUnit, with `@Test` being the main annotation.

If you need to run a `suspend` function, you can use the `kotlin.coroutines.test.runTest` to run the tests inside a `suspend` context.

Tests should follow the naming convention and style guidelines described 🪄Style Coding Standard.

Python

Python Modules should also be tested under `kotlin.test`. You can load the module into a `PythonTestHarness` (WIP) that wraps a `PythonModule`, and then run the tests (WIP)

You can call any method and assertions will automatically propagate and fail the outer test. It will additionally return the value from the method as a `Value`. I plan to add helper methods for this.

If you want to reuse the module, you should initialize it in the class root.

```
class PythonModuleTest {
    val testConsumer = StdoutConsumer(LogLevel.Trace)

    val testHarness by lazy {
      PythonTestHarness(setOf("module.py"))
    }

    @BeforeTest
    fun init() {
        Log.registerConsumer(testConsumer)
    }
```

```
12
13      @AfterTest
14      fun destroy() {
15          Log.revokeConsumer(testConsumer)
16      }
17
18      @Test
19      fun `run test1`() {
20          testHarness.module.methods["test1"]
21            .expect("Module didn't declare a required method
    test1.").execute()
22      }
23
24      @Test
25      fun `run test2`() {
26          val argument = "10"
27          val result = testHarness.module.methods["test2"]
28            .expect("Module didn't declare a required method test2.")
29            .execute(argument)
30          assert(result.asInt() == 10)
31      }
32  }
```

Semi-Manual Tests

The semi-manual tests are modelled the same way as Unit Tests, with one significant difference: These tests have one or more additional requirement that needs to be verified manually.
These tests should be a last-resort in cases where validation would be impossible or computationally very hard. (For example, non-deterministic behaviour or correctness of a generated image)

We label these with the `@SemiManual` marker annotation. I may, in future, it may have checks in place, but for now, you should read the `extraSteps` parameter and when it succeeds, update the `lastSucceeded` value to the current commit hash.

Before merging a branch, these branches should be checked manually.

Model Tests

Due to the fickle and unpredictable nature of machine-learning models, we cannot really test them in a traditional way. These models have to be tested entirely manually from novel data.

Therefore these models have to be tested entirely manually, and then we would have to make the assumption that a model continues to behave as expected.

If possible, we would like to address this gap in testing in the future, but it seems to be an unsolvable problem, that affects the industry at large.

Integration Tests

Integration testing our work is quite difficult. Because of the pipelined nature of our software, the software is very hard to test from end-to-end. As that is the case, our tests of different parts of the pipeline should reflect the behavior of the tests preceeding it.

For example, a test of the CLI that calls the mock-switchboard with a certain set of parameters should have a matching test for the actual switchboard implementation that takes the same arguments.

Since our modules are as idempotent as possible, our tests should be in-practise transitive, and compose to the equivalent of an integration test (Tests for CLI->Switchboard, Switchboard→Module and Module→Expected Result all succeeding imply that CLI->ExpectedResult should also succeed)

Code Coverage

Due to the bi-lingual nature of our codebase, existing dynamic analysis tools cannot accurately determine the test coverage of our codebase. Building our own coverage tool is out-of-scope for a project of this size, so we need to just make our best effort wherever possible.

🤓 Test Plan

Software Testing Goals

The goals of software testing in this project will be classified into two categories: **immediate goals**, involving bug detections and fixes during the developmental phase, and **long-term goals** that focus on overall product quality and user satisfaction after the development.

Immediate Goals

- **Bug Discovery**
  - Identifies errors and failures in developmental phase to improve overall software quality.
- **Bug Prevention**
  - Uses knowledge from detected bugs to avoid repeating the same mistakes in future development.
- **Requirement Validation**
  - Ensures that all functional and non-functional requirements are correctly implemented.
- **Early Defect Detection**
  - Finds issues as early as possible, reducing the cost and effort of refactoring code at later developmental stages.

Long-Term Goals

- **Quality Improvement**
  - Ensures the software is reliable, efficient and accurate (in the eyes of the developers and users).
- **User Satisfaction**
  - Delivers a software that fulfills user expectations and provides a satisfying experience.
- **Reliability**
  - Builds confidence by ensuring the software runs consistently without failures.
- **User Experience Improvement**
  - Ensures the system is simple, intuitive, and user-friendly.
- **Scalability Assurance**
  - Ensures the software can handle increasing workload efficiently.

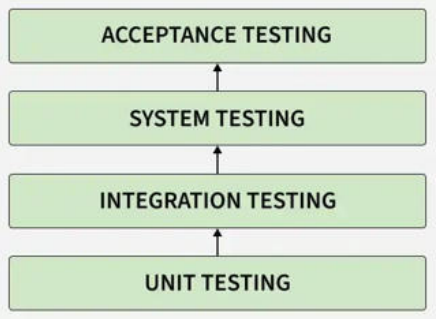
Approaches to Testing: Automated vs Manual

Automated tests are performed by a machine that executes a test script that was written in advance. It is usually more robust and reliable than manual tests. However, due to the stochastic nature of some parts of the software, such as drawing the box segmentations, obtaining the extracted texts via LLMs and review GUIs, some manual tests must be done by qualitatively assessing the output (box segmentation model, image preprocessing, and GUIs for both box segmentation and OCR reviews.

Levels of Software Testing

Software testing is performed at different levels to verify the quality, functionality, and reliability of the product. Each testing level focuses on a specific stage of the software, from individual units to the complete system, thereby identifying issues before the final product is deployed.

The levels of software testing is divided into four main categories as follows:



Source: Levels of Software Testing - GeeksforGeeks

1. Unit Testing

This is the most fundamental level of testing, in which the smallest, functional parts of the software (units) are isolated and tested to validate expected behaviour. Blocks of code that verifies the accuracy of these units are known as **unit tests**, and they will run through Gradle's testing framework and run automatically through GitHub Actions.

Our software exists in a **pipelined nature**, where it can be divided in to different stages. Moreover, there is an **overarching architecture**, called the **Switchboard** that governs the pipeline and manages the workflow and memory, ensuring that the entire process can run smoothly.

Since our software exists in a pipelined nature, some unit tests will be divided by the stage of the pipeline, namely:

[Stage 1] Command-Line Interface (CLI)

Goals:

- Ensure that all commands match with the user's expectation of the software's functionality.
- Ensure that the CLI will parse the commands correctly:
  - Correct commands will be accepted; and
  - Incorrect commands will be rejected.
- Ensure that the correct commands will perform the intended action.

Test method:

Use **automated testing** to assert that each test command will provide its intended output.

[Stage 2] Execute box segmentation

Goal:

- Ensure that the output contains a 4-point polygons for the bounding boxes.

Test method:

Use **automated testing** to ensure that the output for each input images contains coordinates of the vertices of the segmentations.

[Stage 3] Review box segmentation

Goals:

- Ensure that the user can modify the bounding boxes or add new bounding boxes for spine labels and stickers.
- Ensure that the graphical user interface (GUI) is functionable, such that:
  - Users can **modify the bounding boxes** or **add new bounding boxes** for the spine labels and stickers; and
  - Users can navigate between input images with ease.

Test method:

Use **manual testing** to test basic functions, such as:

- Ability to **modify the area** of the segmentations
- Ability to enforce the modifications by **changing the given coordinates**
- Ability to **navigate forward and backward** across images
- Ability to **add new segmentations**

[Stage 4] Text recognition

Goals:

- Ensure that texts can be extracted from an image as *accurately as possible*.
- Ensure that any errors (client-side or server-side) are managed and intercepted gracefully (e.g. with try-except blocks) so that the program wouldn't crash.
- Ensure that the function can be used with any **valid** AI models with a **valid API key**.

Test method:

Use **automated testing** to run tests across several areas, such as:

- *executeOCR*
  - Verify the output when given an invalid API key
  - Verify the output when given an invalid AI model ID
  - Verify the output when tokens has been used up
- *processOutputText*
  - Ensure that the output dictionary for each image will contain only four keys

- Ensure that the corresponding value for each key in the output dictionary will not be empty
- Ensure that the ID strings identified will match the given formats by the client (i.e. regex implemented correctly)

[Stage 5] Review image and structured file

Goals:

- Ensure that the user can review the processed image and the structured output concurrently with ease.
- Ensure that the user can navigate across different images and structured outputs without problems.

Test method:

Use **manual testing** to test basic functions, such as:

- Ability to display the processed image alongside the processed output
- Ability to **navigate forward and backward** across the end results

---

Furthermore, some unit tests will focus on testing smaller units of the Switchboard that are vital to the execution of the software:

Image preprocessing

Goals:

- Ensure that the individual images are segmented as shown in the review.
- Ensure that each individual image can be warped into a rectangle.
- Ensure that the local contrast of each individual image can be enhanced using Contrast Limited Adaptive Histogram Equalisation (CLAHE) to improve text visibility for Stage 4.
- Ensure that images are converted to acceptable file formats for Stage 4.
- Ensure that the stickers are blanked out properly.

Test method:

Use a **combination of automated testing and manual testing**:

- Automated testing
  - Verify that the **extensions of the output files are acceptable** by using OpenAI's API's unsupported image file formats as inputs.
  - Verify that the **dimensions of all images** are consistent (i.e. they are all the same)
- Manual testing
  - Assess the quality and legibility of output images after applying CLAHE

- Verify that the stickers are blanked out.

Export output into a structured file

Goal:

- Ensure that the Python dictionary can be parsed by Kotlin code as an entry row of a CSV file.

Test method:

Use **automated testing**:

- By taking some Python dictionaries, verify that each Python dictionary will correspond to an entry row in the output CSV file.
- Verify that the CSV file is readible.

2. Integration Testing

Integration testing verifies that different modules or services (in our case, different stages of the pipeline) work well together. This test is contingent on the Switcboard being functional. For this software, integration testing will be conducted amongst important interactions in the following manner:

1. CLI, Switchboard, and Box Segmentation Model
2. Box Segmentation Model and Box Segmentation Review
3. Box Segmentation Review, Switchboard (Image preprocessing), and OCR
4. OCR, Switchboard (processing output dictionary), and output display

3. System Testing

Also known as “end-to-end testing”, system testing evaluates the **complete and integrated** software application to verify that it **meets all specified technical, functional and non-functional requirements.** In this case, the system testing will ensure that the software, starting with the CLI, going through Box Segmentation Execution, Box Segmentation Review, OCR, finally ending at Image and Text Review, works without issues. The test will be performed in an environment similar to production.

4. Acceptance Testing

This final level of testing is conducted to determine whether the software is ready for release. During this phase, **the client will test the system** to find out whether the application meets the original business requirements and user expectations. Once this process has been completed and the software has passed all requirements, it will then be acceptable for production.

Code Coverage

Due to the bi-lingual nature of our codebase, existing dynamic analysis tools cannot accurately determine the test coverage of our codebase. Building our own coverage tool is out-of-scope for a

project of this size, so we need to just make our best effort wherever possible.

Changelog

October 25 - *Amy* - Remove section on Injectables.

📈 Test Report

User Story 1 [Xu]

As an archivist, I **need** a usable interface that facilitates the usage of the software.

```

// - These tests cover User Story 1, focusing on CLI usability.
// - The help tests check whether users can find commands and options.
// - Input validation tests check missing arguments, invalid commands,
//   incorrect thresholds and non-existent input directories.

@Test
fun `help lists the main commands`() {
    val result = BaseCommand(MockSwitchboard).test("--help")
    assertEquals(0, result.statusCode)
    listOf("run", "preview", "ocr", "status", "finalise").forEach {
        assertContains(result.output, it)
    }
}

@Test
fun `help explains the important options and subcommands`() {
    val run = BaseCommand(MockSwitchboard).test("run --help")
    assertEquals(0, run.statusCode)
    listOf("--input-directory", "--output", "--no-resume", "--auto-approve-threshold").forEach
{
        assertContains(run.output, it)
    }
    listOf("preview", "ocr").forEach { command ->
        val result = BaseCommand(MockSwitchboard).test("$command --help")
        assertEquals(0, result.statusCode)
        assertContains(result.output, "create")
        assertContains(result.output, "review")
    }
}

@Test
fun `run and finalise require an output path`() {
    listOf("run", "finalise").forEach { command ->
        val result = BaseCommand(MockSwitchboard).test(command)
        assertEquals(1, result.statusCode, command)
        assertContains(result.output, "--output")
    }
}

@Test
fun `invalid commands and thresholds report errors`() {
    listOf("unknown-command", "run -o result.csv -A 101", "preview review -A -1", "preview
review -A abc").forEach { args ->
        val result = BaseCommand(MockSwitchboard).test(args)
        assertEquals(1, result.statusCode, args)
        assertTrue(result.output.isNotBlank(), args)
    }
}

@Test
fun `a missing input directory is rejected`() {
    val result = BaseCommand(MockSwitchboard).test("run -i __missing_archive_directory__ -o
result.csv")
    assertEquals(1, result.statusCode)
}

```

These tests verify that users can find available CLI commands, understand the options, and receive error messages for invalid inputs. This helps ensure the CLI is clear and usable for archivists.

User Story 2 [Uli]

As an archivist, I **need** to automatically identify and separate a shelf of boxes so that I don't have to manually take photos of each.

```
// - getBoxes is run on a sample shelf photo: every box found is returned in order of
//   confidence, and the confidence threshold and edge margin filter out unwanted boxes.
// - Overlapping duplicate detections of the same box are reduced to the most confident one.

val image by lazy { JobRelativePath(directory, Path("PXL_20251127_230150462.jpg")) }

fun rect(x1: Int, y1: Int, x2: Int, y2: Int) =
    Polygon.create(x1 to y1, x2 to y1, x2 to y2, x1 to y2)

@Test
fun getBoxesOrdersBoxesByConfidence() {
    val vertexes = getBoxes(image).second.vertexes!!
    assertTrue(vertexes.isNotEmpty(), "Expected the test image to have boxes")
    assertEquals((1..vertexes.size).map { "box-$it" }.toSet(), vertexes.keys)

    val confidences =
        (1..vertexes.size).map { vertexes.getValue("box-$it").boxBoundary.confidence }
    assertEquals(confidences.sortedDescending(), confidences)
}

@Test
fun getBoxesRespectsConfidenceThreshold() {
    val conf = 0.7
    val boundaries =
        getBoxes(image, conf = conf).second.vertexes!!.values.flatMap {
            it.keyedBoundaries.values
        }
    assertTrue(boundaries.all { it.confidence >= conf })
    // Scores are sigmoids, so nothing can reach above 1
    assertTrue(getBoxes(image, conf = 1.01).second.vertexes!!.isEmpty())
}

@Test
fun getBoxesEdgeMarginDropsBoxes() {
    val all = getBoxes(image, edgeMargin = 0.0).second.vertexes!!.size
    val default = getBoxes(image).second.vertexes!!.size
    assertTrue(default <= all)
    // No box can be centred more than halfway from every side
    assertTrue(getBoxes(image, edgeMargin = 0.5).second.vertexes!!.isEmpty())
}

@Test
fun suppressOverlapsKeepsMostConfidentDuplicate() {
    val kept =
        listOf(
                Detection("Box", 0.6, rect(0, 0, 10, 10)),
                Detection("Box", 0.9, rect(1, 1, 11, 11)),
            )
            .suppressOverlaps()
    assertEquals(listOf(0.9), kept.map { it.score })
}

```

These tests verify that ID stickers are separated from the rest of the box and linked to the correct box, so the sticker can be read on its own. Each test case must pass every assertion to be successful.

User Story 3 [Julius]

As an archivist, I **want** to be able to use the archive's existing unique reference numbers to identify a box, so that I don't have to learn and implement a new reference system.

```
private val service = OCRService(OpenAIAPIData())

// Tests for User Story 3: Unique Reference Number Reading
@ParameterizedTest
@ValueSource(strings = ["1964.0009", "85/7", "UMA-AC-000000056", "UMA-SRE-20160038"])
fun `recognise each existing ID reference format`(id: String) {
    val result = service.processOutputText("Box label\n$id\nDate ranges: None")
    assertEquals(setOf(id), result.idStrings)
}

@Test
fun `reference numbers are removed from description`() {
    val result = service.processOutputText("Australian Pulp and Paper Mills Ltd\n1974.0029,
1974.0030,\n1997.0138\nSome descriptions")
    assertEquals("Australian Pulp and Paper Mills Ltd; Some descriptions", result.description)
}

@Test
fun `duplicate reference numbers collapse to one`() {
    val result = service.processOutputText("[UMA-AC-00000056] Records of\nthe Office of the
Vice-Principal\n(University of Melbourne)\nUMA-AC-00000056")
    assertEquals(setOf("UMA-AC-00000056"), result.idStrings)
}
```

Each test case must pass every line of `assertEquals` to be successful.

User Story 5 [Uli]

As an archivist, I **need** to be able to separate the ID stickers from the 'miscellaneous data' on the rest of the box, so that I can more easily identify which boxes the application refers to.

```
// - Stickers are detected as their own class, so they must not be removed as duplicates of
the
//   box they sit on.
// - Each detected sticker must be assigned to the box that contains it.

val image by lazy { JobRelativePath(directory, Path("PXL_20251127_230150462.jpg")) }

fun rect(x1: Int, y1: Int, x2: Int, y2: Int) =
    Polygon.create(x1 to y1, x2 to y1, x2 to y2, x1 to y2)

@Test
fun suppressOverlapsIgnoresOtherClasses() {
    // A sticker sits inside its box, so must not be suppressed by it
    val kept =
        listOf(
                Detection("Box", 0.9, rect(0, 0, 10, 10)),
                Detection("Sticker", 0.8, rect(0, 0, 10, 10)),
            )
            .suppressOverlaps()
    assertEquals(listOf("Box", "Sticker"), kept.map { it.label })
}

@Test
fun getBoxesStickersSitInsideTheirBox() {
    for (box in getBoxes(image).second.vertexes!!.values) {
        assertEquals(4, box.boxBoundary.points.vertices.size)
        for (sticker in box.labelBoundaries) {
            assertEquals(4, sticker.points.vertices.size)
            assertTrue(box.boxBoundary.points.containsPoint(sticker.points.centroid))
        }
    }
}
```

These tests verify that ID stickers are separated from the rest of the box and linked to the correct box, so the sticker can be read on its own. Each test case must pass every assertion to be successful.

User Story 6 [Julilus]

As an archivist, I **need** to be able to extract text from an isolated archive record.

```
private val service = OCRService(OpenAIAPIData())

// - Each image will be tested based on User Stories 6 and 10.
// - The `assertContains` tests implicitly covers User Story 6 (Extract text from sticker and
//   spine label),since passing them means imply executeOCR() has the ability to extract texts
//   from an isolated image.
// - User Story 8 will not be tested since OCR output isn't deterministic, and what
constitutes
//   as "irrelevant" may differ from one person to another.

@Test
fun `extract info from test image 01`() {
    val image =
        JobRelativePath(
            Path(".").toAbsolutePath().parent,
            Path("src/test/resources/OCRTest/TestImages/clahe_test_01.png"),
        )

    val raw = service.executeOCR(image)
    val output = service.processOutputText(raw)

    assertContains(output.dateRanges.orEmpty(), "1957-01-01:1959-12-31")
    assertEquals(setOf("1974.0029", "1974.0030", "1997.0138"), output.idStrings)
    assertTrue(output.unitNumber == null ||
            output.unitNumber.contains("Unit 1102", ignoreCase = true)
    )
    assertContains(output.description, "Australian Pulp and Paper Mills Ltd", ignoreCase =
true)
    assertContains(output.description, "Materials - paper", ignoreCase = true)
    assertContains(output.description, "to Organization", ignoreCase = true)
    assertContains(output.description, "direct interests", ignoreCase = true)
}

@Test
fun `extract info from test image 02`() {
    val image =
        JobRelativePath(
            Path(".").toAbsolutePath().parent,
            Path("src/test/resources/OCRTest/TestImages/clahe_test_02.png"),
        )

    val raw = service.executeOCR(image)
    val output = service.processOutputText(raw)

    assertContains(output.dateRanges.orEmpty(), "1966-01-01:1971-12-31")
    assertContains(output.dateRanges.orEmpty(), "1966-01-01:1972-12-31")
    assertTrue(
        listOf("1971-01-01:1971-12-31", "1971-01-01:").any { it in
output.dateRanges.orEmpty()}
    )
    assertTrue(
        listOf("1969-01-01:1969-12-31", "1969-01-01:").any { it in output.dateRanges.orEmpty()
}
    )
    assertContains(output.dateRanges.orEmpty(), "1967-01-01:1975-12-31")
    assertContains(output.dateRanges.orEmpty(), "1966-01-01:1975-12-31")
    assertEquals(setOf("UMA-AC-000000056"), output.idStrings)
    assertTrue(output.unitNumber == null ||
            output.unitNumber.contains("Unit 0244", ignoreCase = true)
    )
```

```
    assertContains(output.description, "Press cuttings", ignoreCase = true)
    assertContains(output.description, Regex("1966[\\s]?-[\\s]?71"))
    assertContains(output.description, Regex("1966[\\s]?-[\\s]?72"))
    assertContains(output.description, "1971")
    assertContains(output.description, "1969")
    assertContains(output.description, Regex("1967[\\s]?-[\\s]?75"))
    assertContains(output.description, Regex("1966[\\s]?-[\\s]?75"))
  }

@Test
fun `extract info from test image 03`() {
    val image =
        JobRelativePath(
            Path(".").toAbsolutePath().parent,
            Path("src/test/resources/OCRTest/TestImages/clahe_test_03.png"),
        )

    val raw = service.executeOCR(image)
    val output = service.processOutputText(raw)

    assertContains(output.dateRanges.orEmpty(), "1957-01-01:1958-12-31")
    assertEquals(setOf("1974.0029", "1974.0030", "1997.0138"), output.idStrings)
    assertTrue(output.unitNumber == null ||
            output.unitNumber.contains("Unit 1106", ignoreCase = true)
    )
    assertContains(output.description, "Ballarat Paper Mills P/L", ignoreCase = true)
    assertContains(output.description, "Queensland", ignoreCase = true)
    assertContains(output.description, "Merchants", ignoreCase = true)
}

@Test
fun `extract info from test image 04`() {
    val image =
        JobRelativePath(
            Path(".").toAbsolutePath().parent,
            Path("src/test/resources/OCRTest/TestImages/clahe_test_04.png"),
        )

    val raw = service.executeOCR(image)
    val output = service.processOutputText(raw)

    assertContains(output.dateRanges.orEmpty(), "1957-01-01:1959-12-31")
    assertEquals(setOf("1974.0029", "1974.0030", "1997.0138"), output.idStrings)
    assertTrue(output.unitNumber == null ||
            output.unitNumber.contains("Unit 1108", ignoreCase = true)
    )
    assertContains(output.description, "Ballarat Paper Mills Pty Ltd", ignoreCase = true)
    assertContains(output.description, Regex("1.[\\s]?Buildings[\\s]?-[\\s]?General"))
}

@Test
fun `extract info from test image 05`() {
    val image =
        JobRelativePath(
            Path(".").toAbsolutePath().parent,
            Path("src/test/resources/OCRTest/TestImages/clahe_test_05.png"),
        )

    val raw = service.executeOCR(image)
    val output = service.processOutputText(raw)

    assertTrue(
        listOf("1974-01-01:1974-12-31", "1974-01-01:").any { it in
output.dateRanges.orEmpty()}
    )
    assertTrue(
        listOf("1997-01-01:1997-12-31", "1997-01-01:").any { it in
output.dateRanges.orEmpty()}
    )
    assertEquals(setOf("1974.0029", "1974.0030", "1997.0138"), output.idStrings)
    assertTrue(output.unitNumber == null ||
            output.unitNumber.contains("Unit 1101", ignoreCase = true)
```

```
    )
    assertContains(output.description, "Ballarat Paper Mills Pty Ltd", ignoreCase = true)
    assertContains(output.description, "Shipping Documents", ignoreCase = true)
}
```

For the tests to be successful, the OCR service must be able to extract some texts. The extracted text for each image will then be tested with `assertContains` and `assertEquals` to verify its ability to correctly extract the handwritings. Both `assertContains` and `assertEquals` lines are written based on the ground truths.

Users Story 7 [Julius]

As an archivist, I **want** any identified date-ranges to be labelled in our output to be included additionally to it's appearance in the text, so that I can easily distinguish items of the same collection, where the text may otherwise be long and mostly identical.

```
private val service = OCRService(OpenAIAPIData())

// Tests for User Story 7: Date range identification
@Test
fun `date ranges are extracted and removed from description`() {
    val result = service.processOutputText("Ballarat Paper Mills Pty Ltd\n\nDate ranges: 1957-01-01:1958-12-31")
    assertEquals("Ballarat Paper Mills Pty Ltd", result.description)
}

@Test
fun `multiple date ranges stay comma separated`() {
    val result = service.processOutputText("THE UNIVERSITY OF MELBOURNE\n\nDate ranges: 1974-01-01: , 1974-01-01: , 1997-01-01: , 1957-01-01:1959-12-31")
    assertEquals("1974-01-01: , 1974-01-01: , 1997-01-01: , 1957-01-01:1959-12-31", result.dateRanges)
}

@Test
fun `no date gives null`() {
    val result = service.processOutputText("THE UNIVERSITY OF MELBOURNE\n\nDate ranges: ")
    assertNull(result.dateRanges)
}
```

Each test case must pass every line of `assertEquals` and `assertNull` to be successful.

User Story 8 [Julius]

As a reviewer, I **might** want the ability to detect irrelevant or separable information.

Unfortunately, this user story cannot be effectively tested, as the output of the OCR text extraction is non-deterministic (i.e. every time the OCR services is used for an image, the output will differ).

User Story 10 [Julius]

As a data-scientist, I **want** a well-structured and easy-to-work-with output format so that I can integrate the application with new-and-existing scripts.

The test for this user story is **merged with that of User Story 6**. However, the focus of the tests for this user story mainly revolves around comparing the ground truths against the content of each variable (or field in the business context) of the OCRService class, namely:

- Date ranges (via `output.dateRanges`)

- Unique ID strings (via `output.idStrings`), where all ID strings in the set must be unique
- Unit number (via `output.unitNumber`)
- Description (via `output.description`)*

**Not all lines of the description is tested due to the non-deterministic nature of OCR outputs, so only the ones where the words are clearly written (i.e. handwritings with little to no ambiguity) were tested, as they are predictable.*

User Story 11 [Uli, Amy, Liem]

As an archivist, I **want** an easy-to inspect and edit segmentation output format so that I can review and correct the output easily.

```
// - The segmentation output is written to a single job.json file, which must always be valid
//   JSON so it can be opened and edited by hand or with standard tools.
// - Updating the job only changes the entries that were updated, so corrections to one image
//   don't overwrite the rest.
// - The review GUI test must be performed manually, as it requires interacting with the GUI.
//   The user should adjust the boxes and stickers as if reviewing normally, then verify the
//   printed JSON reflects their changes.

@Test
fun `export json`() = runTest {
    cacheLock.lock("exportJson")
    jsonPath.deleteIfExists()

    JSONJobStateInteraction()
        .create(directory)
        .update(
            baseFiles
                .mapKeys { JobRelativePath(directory, it.key) }
                .mapValues {
                    it.value.run {
                        JobFile(
                            approved = true,
                            vertexes =
                                mutableMapOf(
                                    "box-1" to
                                        ArchiveBox(
                                            Boundary(
                                                Quad.create(0 to 0, 1 to 0, 1 to 1, 0 to 1),
                                                confidence = 1.0,
                                                rawMetadata =
                                                    BoxMetadata(
                                                        null,
                                                        null,
                                                        null,
                                                        description = "test1",
                                                    ),
                                            ),
                                            listOf(
                                                Boundary(
                                                    Quad.create(0 to 0, 1 to 0, 1 to 1, 0 to 1),
                                                    confidence = 0.5,
                                                    rawMetadata =
                                                        BoxMetadata(
                                                            RecordSetID.OldID("1976.0276"),
                                                            1,
                                                            DateRange(
                                                                LocalDate(
                                                                    year = 1971,
                                                                    month = 1,
                                                                    day = 1,
                                                                ),
```

```
                                                                    null,
                                                                ),
                                                                description = "test1",
                                                            ),
                                                        )
                                                    ),
                                                    metadata =
                                                        BoxMetadata(
                                                            RecordSetID.OldID("1976.0276"),
                                                            1,
                                                            DateRange(
                                                                LocalDate(
                                                                    year = 1971,
                                                                    month = 1,
                                                                    day = 1,
                                                                ),
                                                                null,
                                                            ),
                                                            description = "test1;test2",
                                                        ),
                                                )
                                            ),
                                        )
                                    }
                                }
                                .toMutableMap()
                        )

                    JSONJobStateWriterThread.waitWhileWriting()

                    assertDoesNotThrow {
                        Json.parseToJsonElement(
                            jsonPath.readText().also { Log.debug("${jsonPath.pathString} contains \"$it\"") }
                        )
                    }

                    cacheLock.unlock("exportJson")
                }

                @Test
                fun `export json multiple times and ensure correct output`() = runTest {
                    cacheLock.lock("exportMultiJson")
                    jsonPath.deleteIfExists()

                    val state = JSONJobStateInteraction().create(directory)

                    state.update(
                        baseFiles
                            .mapKeys { JobRelativePath(directory, it.key) }
                            .mapValues {
                                it.value.run {
                                    JobFile(
                                        approved = true,
                                        vertexes =
                                            mutableMapOf(
                                                "box-1" to
                                                    ArchiveBox(
                                                        Boundary(
                                                            Quad.create(0 to 0, 1 to 0, 1 to 1, 0 to 1),
                                                            confidence = 1.0,
                                                            rawMetadata =
                                                                BoxMetadata(
                                                                    null,
                                                                    null,
                                                                    null,
                                                                    description = "test1",
                                                                ),
                                                        ),
                                                        listOf(
                                                            Boundary(
                                                                Quad.create(0 to 0, 1 to 0, 1 to 1, 0 to 1),
                                                                confidence = 0.5,
```

```
                                                rawMetadata =
                                                    BoxMetadata(
                                                        RecordSetID.OldID("1976.0276"),
                                                        1,
                                                        DateRange(
                                                            LocalDate(
                                                                year = 1971,
                                                                month = 1,
                                                                day = 1,
                                                            ),
                                                            null,
                                                        ),
                                                        description = "test1",
                                                    ),
                                            )
                                        ),
                                        metadata =
                                            BoxMetadata(
                                                RecordSetID.OldID("1976.0276"),
                                                1,
                                                DateRange(
                                                    LocalDate(year = 1971, month = 1, day = 1),
                                                    null,
                                                ),
                                                description = "test1;test2",
                                            ),
                                    )
                                ),
                            )
                        }
                    }
                    .toMutableMap()
            )
            state.update(
                baseFiles
                    .mapKeys { JobRelativePath(directory, it.key) }
                    .mapValues {
                        it.value.run {
                            JobFile(
                                approved = true,
                                vertexes =
                                    mutableMapOf(
                                        "box-1" to
                                            ArchiveBox(
                                                Boundary(
                                                    Quad.create(0 to 0, 100 to 0, 100 to 100, 0 to 100),
                                                    confidence = 1.0,
                                                    rawMetadata =
                                                        BoxMetadata(
                                                            null,
                                                            null,
                                                            null,
                                                            description = "test1",
                                                        ),
                                                ),
                                                listOf(
                                                    Boundary(
                                                        Quad.create(
                                                            0 to 0,
                                                            100 to 0,
                                                            100 to 100,
                                                            0 to 100,
                                                        ),
                                                        confidence = 0.5,
                                                        rawMetadata =
                                                            BoxMetadata(
                                                                RecordSetID.OldID("1976.0276"),
                                                                1,
                                                                DateRange(
                                                                    LocalDate(
```

```
                                                                    year = 1971,
                                                                    month = 1,
                                                                    day = 1,
                                                                ),
                                                            ),
                                                        null,
                                                    ),
                                                description = "test1",
                                            ),
                                        )
                                    ),
                                )
                            ),
                        )
                    }
                }
                .filter { it.key.relative.pathString.contains("2311") }
                .toMutableMap()
        )

        JSONJobStateWriterThread.waitWhileWriting()

        val json = assertDoesNotThrow { Json.parseToJsonElement(jsonPath.readText()) }

        assertNotNull(
            json.jsonObject[baseFiles.keys.first { it.pathString.contains("2311") }.pathString]
                ?.jsonObject["vertexes"]
                ?.toStrictString()
                ?.contains("{\"x\":100,\"y\":100}")
                .takeIf { true }
        )

        assertNotNull(
            json.jsonObject[baseFiles.keys.first { !it.pathString.contains("2311") }.pathString]
                ?.jsonObject["vertexes"]
                ?.toStrictString()
                ?.contains("{\"x\":1,\"y\":1}")
                .takeIf { true }
        )

        cacheLock.unlock("exportMultiJson")
    }

    @SemiManual(
        "Manually Interact with the GUI.",
        lastSucceeded = "bfb9ca7fe545ce26860539588340ef886e373927",
    )
    @DisabledIfEnvironmentVariable(named = "GITHUB_ACTIONS", matches = "true")
    @Test
    fun `try review interaction`() {
        val files =
            mapOf(
                    Path("PXL_20251127_230150462.jpg") to
                        JobFile(
                            vertexes =
                                mutableMapOf(
                                    "box-1" to
                                        ArchiveBox(
                                            Boundary(
                                                points =
                                                    Quad.create(
                                                        230 to 10,
                                                        2350 to 130,
                                                        2170 to 500,
                                                        260 to 360,
                                                    ),
                                                confidence = 0.08,
                                            ),
                                            listOf(
                                                Boundary(
                                                    points =
                                                        Quad.create(
                                                            1700 to 150,
```

```
                                                    2180 to 250,
                                                    2140 to 310,
                                                    1700 to 300,
                                                ),
                                            confidence = 0.90,
                                        )
                                    ),
                                )
                            )
                        ),
                    Path("PXL_20251127_230207837.jpg") to
                        JobFile(
                            vertexes =
                                mutableMapOf(
                                    "box-1" to
                                        ArchiveBox(
                                            Boundary(
                                                points =
                                                    Quad.create(
                                                        154 to 860,
                                                        1785 to 1113,
                                                        1789 to 1600,
                                                        488 to 1581,
                                                    ),
                                                confidence = 1.0,
                                            ),
                                            listOf(
                                                Boundary(
                                                    points =
                                                        Quad.create(
                                                            1144 to 1493,
                                                            1421 to 1458,
                                                            1390 to 1282,
                                                            1133 to 1293,
                                                        ),
                                                    confidence = 0.90,
                                                )
                                            ),
                                        )
                                )
                        ),
                )
                .mapKeys { JobRelativePath(directory, it.key) }
                .toMutableMap()
        println(
            SegmentationGUIInteraction().review(files).mapValues { (_, v) ->
                Json.encodeToString(v)
            }
        )
}

```

These tests verify that the segmentation output is saved as a valid JSON file that can be inspected and edited directly, that updates only change the entries they apply to, and that the output can be reviewed and corrected in the GUI. Each test case must pass every assertion to be successful.

User Story 12 [Uli]

As a reviewer, I **want** to know which entries are least coherent, so that I can prioritise them for review.

```
// - An entry's coherence is the confidence of its least confident detection (box or sticker),
//   so a single doubtful detection marks the whole entry as needing review.
// - Entries are sorted by this score before review, so the least coherent are shown first.
//   Entries with no detections at all are ranked first.

```

```
fun boundary(confidence: Double) =
    Boundary(Quad.create(0 to 0, 10 to 0, 10 to 10, 0 to 10), confidence)

fun jobFile(vararg boxes: ArchiveBox) =
    JobFile(boxes.mapIndexed { i, box -> "box-${i + 1}" to box }.toMap().toMutableMap())

@Test
fun minConfidenceIsLeastConfidentBoundary() {
    val file =
        jobFile(
            ArchiveBox(boundary(0.9), listOf(boundary(0.8))),
            ArchiveBox(boundary(0.7), listOf(boundary(0.3), boundary(0.95))),
        )
    assertEquals(0.3, file.minConfidence)
    // Files with no detections are treated as the least coherent
    assertEquals(-999.0, JobFile().minConfidence)
}

@Test
fun leastCoherentFilesAreReviewedFirst() {
    val files: Map<String, JobFile?> =
        mapOf(
            "confident.jpg" to jobFile(ArchiveBox(boundary(0.9), listOf(boundary(0.85)))),
            "unsure.jpg" to jobFile(ArchiveBox(boundary(0.9), listOf(boundary(0.4)))),
            "unprocessed.jpg" to null,
            "empty.jpg" to JobFile(),
        )
    // Same ordering as SegmentationGUIInteraction.review
    val order = files.entries.sortedBy { it.value?.minConfidence ?: -999.0 }.map { it.key
}
    assertEquals(listOf("unprocessed.jpg", "empty.jpg", "unsure.jpg", "confident.jpg"),
order)
}
```

These tests verify that each entry is given a coherence score based on its least confident detection, and that entries are ordered so the least coherent are reviewed first. Each test case must pass every assertion to be successful.

User Story 13 [Liem]

As a reviewer, I **want** to be able to review the generated bounding boxes (or the resultant images), so that I can adjust any that don't align properly.

Test ID: SEG_GUI - 01

Test Scenario: Preview all images

Test Steps:

1. Press "next" button until at final image in batch, then keep pressing it.
2. Press "previous" button until at final image in batch, then keep pressing it.

Expected Result: Should preview every image in batch, loading every image and all corresponding metadata in the table. Step 1 "next" presses on the final image should not do anything and "previous" presses on the first image should not do anything. File counter should be accurate to image in batch.

Actual Result: Expected

Status: Pass

Test ID: SEG_GUI - 02

Test Scenario: Viewport zoom controls

Test Steps:

1. Continuously press “+“ button
2. Continuously press “-“ button
3. Press “Fit“ button

Expected Result: Step 1 should reach a maximum zoom in and step 2 should reach a maximum zoom out. Step 3 should restore zoom to initial state.

Actual Result: Expected

Status: Pass

Test ID: SEG_GUI - 03

Test Scenario: Preview saving and finish

Test Steps:

1. Click on an existing segmentation to select it.
2. Drag the box area to another position.
3. Click the “confirm“ button.
4. Click on another segmentation and press the “delete” button.
5. Press “next“ button and then “previous“ button.
6. Press “finish“ button.

Expected Result: Selection is made (with highlight and vertexes). Movement moves the box, sticker, and selection. After pressing “previous“ button in step 4, changes should still be visible. Finish updates json file.

Actual Result: Expected

Status: Pass

Test ID: SEG_GUI - 04

Test Scenario: Unsaved changes

Test Steps:

1. Click on an existing segmentation to select it.
2. Drag the box area to another position.

3. Click the “next“ button without saving and then click “previous” button
4. Click “create” button
5. Click the “next“ button without saving and then click “previous” button
6. Click “finish“ button

Expected Result: None of the modifications made should be visible in preview when clicking “previous“ in steps 3 and 5 and the json output should identical to the input.

Actual Result: Expected

Status: Pass

Test ID: SEG_GUI - 05

Test Scenario: Window scaling

Test Steps:

1. While not in fullscreen, drag window wider horizontally.
2. Scale back to initial position.
3. Drag window taller vertically.
4. Scale back to initial position
5. Drag window larger diagonally.
6. Drag window smaller diagonally
7. Scale back to initial position.

Expected Result: Image viewport should always be displays at a 3:4 scale. Text and buttons should remain at the same size. Image scale should remain at the same zoom. No elements should overlap. All buttons above and below the viewport should be aligned horizontally. Image viewport and table should be aligned horizontally. File counter should remain in bottom right.

Actual Result: Expected

Status: Pass

Test ID: SEG_GUI - 06

Test Scenario: Vertex movement

Test Steps:

1. Select an existing segmentation.
2. Drag the top-left vertex of the box selection to the top-right of the image viewport, trying to drag it out.

3. Drag the top-left vertex of the sticker selection to the top-right of the image viewport, trying to drag it out.
4. Press “confirm“ button.
5. Press “finish“ button.

Expected Result: Both vertexes should move, along with the highlighted selection area. Both vertex should be clamped in the bounds of the image viewport and the output should match the vertex position. The sticker vertex should not be clamped in bounds of the box selection.

Actual Result: Expected

Status: Pass

Test ID: SEG_GUI - 07

Test Scenario: Mouse dragging with selection

Test Steps:

1. Drag mouse outside the area of any segmentation.
2. Drag mouse within the area of an unselected segmentation.
3. Drag mouse outside the area of any segmentation
4. Drag mouse within the area of another (unselected) segmentations' sticker.
5. Drag mouse within the area of the selected segmentations' sticker.
6. Click outside the area of any segmentation.

Expected Result: For step 1, the image viewport should pan. For step 2, the segmentation should be selected and the drag moves the entire selection. For step 3, the selection should remain and the image viewport should pan. For step 4, the entire selection of the segmentation should move. For step 5, just the sticker of the selection should move. For step 6, the selection should be deselected.

Actual Result: Expected

Status: Pass

Test ID: SEG_GUI - 08

Test Scenario: Box and sticker movement

Test Steps:

1. Select an existing segmentation.
2. Try to drag the box area out of the image viewport.
3. Try to drag the sticker out of the box (not against image viewport).
4. Try to drag the sticker out of the image viewport.

Expected Result: For step 2, none of the selected box's area should be able to leave the viewport. For step 3, the sticker should not be able to be dragged out of the box selection area (in a quadrilateral bound by the furthest points). For step 4, none of the selected sticker's area should be able to leave the viewport.

Actual Result: Expected

Status: Pass

Test ID: SEG_GUI - 09

Test Scenario: Creation

Test Steps:

1. Press the "create" button.
2. Move the selection.
3. Move the top-left vertex anywhere else.
4. Press the "confirm" button.
5. Press the "finish" button.

Expected Result: Created segmentation should be visible on image viewport. A new, corresponding record should be added to the json file output.

Actual Result: Expected

Status: Pass

User Story 15 [Amy]

As a reviewer, I **want** to be able to merge and easily deduplicate the results of multiple runs, so that I can easily segment my work.

```
    @Test
    fun `only process files valid for a given step`() = runTest {
            val consumer = StringConsumer(LogLevel.Trace)
            Log.registerConsumer(consumer)

            cacheLock.lock("merge_runs")
            val cacheGuard = cache.writeLock()
            val cache = cacheGuard.value

            cache.put(
                directory,
                ConcreteJobState(
                    directory,
                    starterState =
                        baseFiles
                            .mapValues {
                                it.value.run {
                                    JobFile(/* A JobFile definition that would be ready to
export. */)
                                }
                            }
                ),
            )
```

```
            cache.get(directory)!!.update(mapOf(JobRelativePath(directory,
Path("PXL_20251127_230317928.jpg")) to JobFile(vertexes = mutableMapOf("box-1" to
                ArchiveBox(
                    Boundary(
                        Quad.create(0 to 0, 1 to 0, 1 to 1, 0 to 1),
                        confidence = 1.0,
                    ),
                    listOf(
                        Boundary(
                            Quad.create(0 to 0, 1 to 0, 1 to 1, 0 to 1),
                            confidence = 0.5,
                        )
                    ),
                )))))

            cacheGuard.release()

            MockSwitchboard.reviewSegmentationPreview(directory)

            cacheLock.unlock("merge_runs")
            Log.revokeConsumer(consumer)
            val log = consumer.getLog()

            assertContains(log, "1 total files, 0 above auto-approval threshold 1.0")
        }
```

This code ensures that the user's calls to the switchboard do not pointlessly repeat work that has already been done.

User Story 16 [Liem]

As a reviewer, I **might** want an interface, so that OCR output can be manually adjusted.

Test ID: OCR_GUI - 01

Test Scenario: Preview all images

Test Steps:

1. Press "next" button until at final image in batch, then keep pressing it.
2. Press "previous" button until at final image in batch, then keep pressing it.

Expected Result: Should preview every image in batch, loading every image and all corresponding metadata in the table. Step 1 "next" presses on the final image should not do anything and "previous" presses on the first image should not do anything. File counter should be accurate to image in batch.

Actual Result: Expected

Status: Pass

Test ID: OCR_GUI - 02

Test Scenario: Viewport zoom controls

Test Steps:

1. Continuously press "+" button
2. Continuously press "-" button

3. Press “Fit“ button

Expected Result: Step 1 should reach a maximum zoom in and step 2 should reach a maximum zoom out. Step 3 should restore zoom to initial state.

Actual Result: Expected

Status: Pass

Test ID: OCR_GUI - 03

Test Scenario: Text preview

Test Steps:

1. Type text in “Node Title” field until goes off screen
2. Delete typed text and repeat step 1 for “Description“ and “Date Range“ fields
3. Type text in all 3 fields to go off screen

Expected Result: Preview properly and instantly updates according to input text. When text goes off screen for any field, scroll bar should appear on the right of the table, slightly realigning the table. For step 3, when the text that goes the furthest is what should determine the scrollbar scale.

Actual Result: Expected

Status: Pass

Test ID: OCR_GUI - 04

Test Scenario: Input field

Test Steps:

1. Type text until preview reaches edge.
2. Add spaces after the text and add more text.
3. Add spaces until preview reaches edge.
4. Add text after the spaces.
5. Paste in text including a newline character.

Expected Result: Preview should wrap text, breaking words into multiple lines if exceeding the line width, including spaces. Input field should never break into multiple lines. Caret should always be aligned with text for input field (excluding spaces).

Actual Result: Expected

Status: Pass

Test ID: OCR_GUI - 05

Test Scenario: Window scaling

Test Steps:

1. Type one line of text in “Description” input field
2. While not in fullscreen, drag window wider horizontally.
3. Scale back to initial position.
4. Drag window taller vertically.
5. Scale back to initial position
6. Drag window larger diagonally.
7. Drag window smaller diagonally
8. Scale back to initial position.

Expected Result: Image viewport should always be displays at a 3:4 scale. Text and buttons should remain at the same size. Text should update wraps. Image scale should remain at the same zoom. No elements should overlap. All buttons should be aligned horizontally. Image viewport and table should be aligned horizontally. Vertical scrollbar should appear for step 6 and disappear for step 7. File counter should remain in bottom right.

Actual Result: Expected

Status: Pass

Test ID: OCR_GUI - 06

Test Scenario: Save and finish

Test Steps:

1. Replace any existing text with “a” in “Description” field.
2. Press “save“ button.
3. Replace “a“ with “b“.
4. Press “next“ button and “previous“ button without saving.
5. Press “finish“ button.

Expected Result: Description field at the end of step 4 should read “a“. GUI should output “a“ for JobFile description field.

Actual Result: Expected

Status: Pass

Test ID: OCR_GUI - 07

Test Scenario: Validate “Node Title” field

Test Steps:

1. Type “1976 Unit 71“ into “Node Title“ field.
2. Press “save“ button.
3. Type “1976 U 1071“ into “Node Title“ field.
4. Press “save“ button
5. Press “ok” button on error popup
6. Type “1976 Unit“ into “Node Title“ field.
7. Press “save“ button
8. Press “ok” button on error popup
9. Type “Unit 1071“ into “Node Title“ field.
10. Press “save“ button
11. Press “ok” button on error popup
12. Press “finish“ button

Expected Result: The save for step 2 should be successful and change input field from “1976 Unit 71“ to “1976 Unit 0071“. Error popup should appear after steps 3, 6, and 9. The “ok“ button should close the error popup each time. Finish should show “1976 Unit 0071“ for GUI output.

Actual Result: Expected

Status: Pass

Test ID: OCR_GUI - 08

Test Scenario: Validate “Date Range” field

Test Steps:

1. Type “1999;;“ into “Date Range“ field.
2. Press “save” button.
3. Type “1999-2000;1999-01-01;2000-1-1“ into “Date Range“ field.
4. Press “save” button.
5. Type “2000-1999;1999-01-01;2000-01-01“ into “Date Range“ field.
6. Press “save” button.
7. Press “ok“ button on error popup.
8. Type “1999-2000;2000-01-01;1999-01-01“ into “Date Range“ field.
9. Press “save” button.
10. Press “ok“ button on error popup.
11. Type “1999-2000;1999-02-31;2000-01-01“ into “Date Range“ field.

12. Press “save” button.
13. Press “ok“ button on error popup.
14. Press “finish“ button

Expected Result: The save for step 2 should be successful and change input field from “1999-2000;1999-01-01;2000-01-01“. Error popup should appear after steps 7, 10, and 13. The “ok“ button should close the error popup each time. Finish should show “1999-2000;1999-01-01;2000-01-01“ for GUI output.

Actual Result: Expected

Status: Pass

Test ID: OCR_GUI - 09

Test Scenario: Joint error messages

Test Steps:

1. Enter “a” into “Node Title” field.
2. Enter “a“ into “Date Range“ field.
3. Press “save“ button.

Expected Result: Error popup should appear on step 3 with two distinct error messages.

Actual Result: Expected

Status: Pass

Test ID: OCR_GUI - 10

Test Scenario: Empty input fields

Test Steps:

1. Enter “” into all three fields.
2. Press “save” button.
3. Press “finish“ button.

Expected Result: No error popup should appear. JobFile output should have empty metadata for the ArchiveBox.

Actual Result: Expected

Status: Pass

User Story 17 [Amy]

As an archivist, I **might** want the ability to aggregate multiple images to help with legibility.

This functionality is a trivial usage of standard library functions and therefore doesn't warrant a unit test.

User Story 18 [Amy]

As a user, I **want** a single cohesive application that can work from end-to-end, so that I'm not downloading or trying to fix several disconnected tools.

Tested in SwitchboardEndToEnd. This test must be performed manually as it interacts with the system configuration and the GUIs, which cannot be automated.
The user should simply interact with each component as if they are using the software as normal, and then verify the OCR output CSV.

User Story 19 [Uli]

As a data-scientist, I **might want** be able to tweak the Machine Learning components in a familiar language, so that I can adjust the model if it isn't performing as expected.

```
// - The model is trained and exported to ONNX in Python (model-sandbox/YOLO-train.py and
//   model-export.py) with Ultralytics, so it can be retrained without touching the Kotlin code.
// - getBoxes accepts any exported model through modelPath, reading its class names and input
//   size from the export metadata, and falls back to the bundled model when none is given.

val image by lazy { JobRelativePath(directory, Path("PXL_20251127_230150462.jpg")) }

@Test
fun copyModelIsCachedAndExists() {
    val first = BoxSegmentation.copyModelFromResourcesIfNotExists(DEFAULT_MODEL_RESOURCE)
    val second = BoxSegmentation.copyModelFromResourcesIfNotExists(DEFAULT_MODEL_RESOURCE)
    assertTrue(first.exists())
    assertEquals(first, second)
}

@Test
fun getBoxesUsesSuppliedModel() {
    // Stands in for a model retrained and re-exported from Python
    val exported = createTempDirectory().resolve("retrained.onnx")

    BoxSegmentation.copyModelFromResourcesIfNotExists(DEFAULT_MODEL_RESOURCE).copyTo(exported)

    val custom = getBoxes(image, modelPath = exported).second.vertexes!!
    val default = getBoxes(image).second.vertexes!!
    assertTrue(custom.isNotEmpty(), "Expected the supplied model to find boxes")
    assertEquals(default.keys, custom.keys)
    assertEquals(
        default.values.map { it.boxBoundary.confidence },
        custom.values.map { it.boxBoundary.confidence },
    )
}
```

These tests verify that the segmentation model can be retrained and exported in Python, then used by the software in place of the bundled model without any changes to the Kotlin code. Each test case must pass every assertion to be successful.

User Story 20 [Amy, Xu]

As a data-scientist, I might want to use a part of the software pipeline as a standalone tool, so that I can use it for other applications.

```
1
2  // - These tests cover the CLI part of User Story 20.
3  // - Preview creation and OCR review are tested as separate commands.
4  // - The finalise test checks that a CSV file can be generated
5  //   from an existing job without running the entire pipeline.
6  // - MockSwitchboard is used to avoid running real image processing.
7
8  @Test
9  fun `preview create can be called separately`() = withSampleImage { directory, _ ->
10     var calls = 0
11     val preview = object : CreatePreviewInteraction {
12         override fun createPreview(
13             files: Set<JobRelativePath>,
14             weights: Path?,
15             rejectConfidenceThreshold: Double,
16         ): Map<JobRelativePath, JobFile> {
17             calls++
18             assertEquals(1, files.size)
19             return emptyMap()
20         }
21     }
22     val switchboard = MockSwitchboard.with(preview)
23     val result = BaseCommand(switchboard).test("preview", "create", "-i", directory.toString())
24     assertEquals(0, result.statusCode, result.output)
25     assertEquals(1, calls)
26 }
27
28 @Test
29 fun `OCR review can be called separately`() = withSampleImage { directory, image ->
30     var calls = 0
31     val review = object : OCRPreviewInteraction {
32         override fun review(
33             files: Map<JobRelativePath, JobFile?>,
34             autoApproveThreshold: Double,
35         ): Map<JobRelativePath, JobFile> {
36             calls++
37             return emptyMap()
38         }
39     }
40     val switchboard = MockSwitchboard.with(review)
41     val state = requireNotNull(switchboard.job.create(directory))
42     state.update(mapOf(JobRelativePath(directory, image.fileName) to JobFile(vertexes =
   mutableMapOf())))
43
44     val result = BaseCommand(switchboard).test("ocr", "review", "-i", directory.toString())
45     assertEquals(0, result.statusCode, result.output)
46     assertEquals(1, calls)
47 }
48
49 @Test
50 fun `finalise writes a CSV from an existing job`() = withSampleImage { directory, image ->
51     val output = directory.resolve("result.csv")
52     try {
53         val switchboard = MockSwitchboard.with()
54         val state = requireNotNull(switchboard.job.create(directory))
55         state.update(mapOf(JobRelativePath(directory, image.fileName) to JobFile(vertexes =
   mutableMapOf())))
56         val result = BaseCommand(switchboard).test("finalise", "-i", directory.toString(), "-
   o", output.toString())
57         assertEquals(0, result.statusCode, result.output)
58         assertTrue(Files.exists(output))
59         assertTrue(Files.size(output) > 0)
60     } finally {
61         Files.deleteIfExists(output)
62     }
63 }
64
```

These tests verify that individual processing stages can be accessed separately through the CLI, including preview creation, OCR review, and CSV export. This supports users who only need specific

parts of the pipeline.

User Story 21 [Liem, Xu]

As a developer, I **need** the architectural workflow of the CLI commands defined and validated, so that I can implement the CLI consistently with the system architecture.

```

// - These tests cover part of User Story 21, focusing on
//   the integration between CLI commands and Switchboard.
// - The status test checks whether the selected Switchboard
//   is used to retrieve job information.
// - The preview review test checks whether CLI parameters
//   are correctly passed to the corresponding interaction.

@Test
fun `status uses the selected Switchboard`() {
    val status = StringStatusInteraction()
    val switchboard = MockSwitchboard.with(status)
    val directory = Files.createTempDirectory("cli-status-")
    try {
        val result = BaseCommand(switchboard).test("status", "-i", directory.toString())
        assertEquals(0, result.statusCode, result.output)
        assertContains(status.result, "no active job")
    } finally {
        Files.deleteIfExists(directory)
    }
}

@Test
fun `preview review passes the threshold to Switchboard`() = withSampleImage { directory, image
->
    var threshold: Double? = null
    val review = object : SegmentationPreviewInteraction {
        override fun review(
            files: Map<JobRelativePath, JobFile?>,
            autoApproveThreshold: Double,
        ): Map<JobRelativePath, JobFile> {
            threshold = autoApproveThreshold
            return emptyMap()
        }
    }
    val switchboard = MockSwitchboard.with(review)
    val state = requireNotNull(switchboard.job.create(directory))
    state.update(mapOf(JobRelativePath(directory, image.fileName) to JobFile(vertexes =
mutableMapOf())))

    val result = BaseCommand(switchboard).test("preview", "review", "-i", directory.toString(),
"-A", "25")
    assertEquals(0, result.statusCode, result.output)
    assertEquals(0.25, threshold)
}

```

These tests verify that CLI commands interact with the correct Switchboard components and pass parameters correctly. This helps ensure the CLI follows the intended system architecture.

User Story 23 [Uli]

As an archivist, I **want** the system to skip OCR on boxes without any sticker, so that processing time isn't wasted on out-of-scope boxes.

```
// - A sticker is assigned to the box containing its centre. A sticker centred just outside
every
//   box (e.g. on an edge) goes to the sticker-less box it overlaps most, so boxes with a
//   sticker are not mistaken for out-of-scope ones.
```

```
// - Stickers overlapping no box are dropped, as their box is out of frame, and boxes with no
//   sticker nearby are left without one, so they can be skipped.

fun rect(x1: Int, y1: Int, x2: Int, y2: Int) =
    Polygon.create(x1 to y1, x2 to y1, x2 to y2, x1 to y2)

@Test
fun toArchiveBoxesAssignsStickerOutsideBoxToOverlappingBox() {
    val boxes =
        listOf(
                Detection("Box", 0.9, rect(0, 0, 100, 100)),
                Detection("Box", 0.8, rect(200, 0, 300, 100)),
                Detection("Sticker", 0.7, rect(40, 40, 60, 60)),
                // Centred just past the right edge of the second box
                Detection("Sticker", 0.6, rect(290, 40, 320, 60)),
            )
            .toArchiveBoxes()
    assertEquals(listOf(0.7), boxes.getValue("box-1").labelBoundaries.map { it.confidence })
    assertEquals(listOf(0.6), boxes.getValue("box-2").labelBoundaries.map { it.confidence })
}

@Test
fun toArchiveBoxesOnlyGivesUnassignedStickersToStickerlessBoxes() {
    val boxes =
        listOf(
                Detection("Box", 0.9, rect(0, 0, 100, 100)),
                Detection("Box", 0.8, rect(200, 0, 300, 100)),
                Detection("Sticker", 0.7, rect(40, 40, 60, 60)),
                // Overlaps the first box, which already has a sticker
                Detection("Sticker", 0.6, rect(95, 40, 120, 60)),
                // Both overlap the second box, so the more confident wins
                Detection("Sticker", 0.5, rect(290, 40, 320, 60)),
                Detection("Sticker", 0.4, rect(290, 70, 320, 90)),
                // Overlaps no box, so belongs to one out of frame
                Detection("Sticker", 0.3, rect(500, 500, 510, 510)),
            )
            .toArchiveBoxes()
    assertEquals(listOf(0.7), boxes.getValue("box-1").labelBoundaries.map { it.confidence })
    assertEquals(listOf(0.5), boxes.getValue("box-2").labelBoundaries.map { it.confidence })
    // A box with no sticker nearby is left without one
    assertEquals(
        listOf<Double>(),
        listOf(Detection("Box", 0.9, rect(0, 0, 100, 100)))
            .toArchiveBoxes()
            .getValue("box-1")
            .labelBoundaries
            .map { it.confidence },
    )
}
```

These tests verify that every box with a sticker on or near it is given that sticker, and that boxes without one are left empty so they can be identified as out of scope. Each test case must pass every assertion to be successful.

User Story 24 [Amy]

As a user, I **need** a program that implements all of the functionality it claims to provide, so that I can trust the program to successfully complete my work.

This is tested via. the tests for the individual components as described into the earlier user stories.

Project Structure

Software Structure

Switchboard

The project uses an adapter pattern to simplify dependency and allow for easy swapping of implementations, using the `Switchboard` object (and `AbstractSwitchboard` open `class`) as the master orchestrator and dependency manager. Which allows for us to seamlessly experiment with different implementations of features, for testing and to help us mock certain tasks (eg. the Job file can exist in memory if we are using the `run` sub-command).

Everything that the program can do is embodied by an interface that extends the sealed `Interaction` class. The inheriting interfaces are not sealed, allowing for custom implementations of existing interaction types, but not for custom interaction types. This also allows for us to define common interaction behavior, like composition of multiple interactions where this makes sense (`InitInteraction`s, for example, need to compose multiple to encompass the expected behavior).

These interactions map closely to CLI subcommands, but with a few extra used to define common behavior, like managing the Job state. In fact, the CLI calls the functions on the switchboard, that immediately delegate to the relevant `Interaction`s, potentially with some additional logic to handle things like job state.

Python Interactions

These interactions exist as a Python Module, adapted to feel like normal Kotlin code.
The classes used for this purpose are documented in Using Python Code from Kotlin

Box Detection

Box detection is handled as a self-contained python package with a set of functions corresponding to interactions with the model.

- `loadModel(weightsPath)` - Loads the trained YOLO model and caches it by weights path, so it can be warmed once at startup rather than reloaded on every call.
- `detectDirectory(sourceDir, weightsPath, conf)` - The main entry point for a batch run. Walks a directory of images, runs detection on each, and returns results shaped to match the Job State schema (four corner points, a type, and a placeholder for transcription). Non-image files are recorded rather than skipped, so later runs don't treat them as new.
- `getCornerPoints(sourceDir, weightsPath, conf)` - A coordinates-only view of detections, stripped of type and transcription fields. Intended for feeding straight into the

corner-correction UI before the full Job State structure is wired up.
- `refineCorners(imagePath, cropId, newPoints)` - The write-back point for the Review step. Takes a set of user-adjusted corner points and returns them in Job State crop format so the correction can be saved.

OCR

Other Interactions

Init

We have a few behaviors that should run regardless of the subcommand, they fall under 'init' interactions.

Logger

Globally, including within our Python modules, we have a unified Logging system, built from scratch. It can log to multiple recievers and is somewhat configurable. While it mostly exists for debugging, allowing us to see where execution goes wrong, it is also used to report messages about errors and warnings to the end user. It is available within the `Log` class in either language and is injected via. the `Logger` injectable. It is initialised regardless of subcommand and can be used as an `InitInteraction`

Job State Handling

We store our state between steps in a 'Job State'. This interaction is designed to be handed around to other interactions and provides ways of accessing and modifying the job state, potentially as a file.

The job state format is described in detail in Segmentation Output File Format

We intend for the Job State to easily support multiple formats:

Memory

When doing a full run in the one command, using `run` (which we will need to consider if resuming is important), or testing; we do not need to actually write this data to a file. Instead, we can simply store it inside a `data class`, saving the IO overhead.

JSON

The main format for the process we have planned to use from the beginning is JSON. This is the "canonical" format.

Others

We have some minor thoughts about options for more performant state handling, like SQLite, these are out of scope for the current project, but might be a good idea for the future.

Converting

When we introduce other formats, we plan to allow for conversion from format to format on a required basis.

The biggest hurdle here is format detection, since we can defer the normal operations to the origin format. The way that we manage this, is that we make every Companion implement an interface and register itself in an object.

Review

The review action allows the end-user to adjust and correct boxes detected b Project Stru cture | Box Detection The current intended implementation is to use a small GUI application by default to review the boxes, and add a flag to effectively 'skip' this step.

Crop

May be renamed in future because is really all image manipulation between Box Detection and OCR.

Export

Once we have processed everything, we must export it to a CSV, the client's data-format of choice. There is room in place for implementing other types of exporter, but no plans to do so.

Frontend

While the CLI is currently the only frontend, the frontend code is as decoupled as possible from the backend, having only a few classes of interaction surface. One of these is `Log`, which exists globally. Another is the `Switchboard`, which is the primary adapter.

One switchboard item, Project Structure | Review, also requires a small GUI frontend, for a single pipeline task.

In theory, our design this would help if implementing a fully-fledged GUI in the future, as our business logic is very detached from the CLI - however, a GUI of this scale is out of our scope for the project.

File Structure

The repository has quite a substantial folder structure. A lot of the layout is by convention from Gradle.

Below is a subset of the file tree, and what each part is for.

```
.
├── app -- Files relating to the CLI frontend.
│   ├── build
│   │   ├── classes -- Generated JVM classes
│   │   ├── distributions -- Zip Bundles including all Java Dependencies for app.jar
│   │   ├── kotlin -- Kotlin Compiler Intermediary Data (for caching certain computations)
│   │   ├── ktfmt -- Summaries of any KtFmt runs
│   │   ├── libs -- The generated executable JAR file
│   │   ├── scripts -- Auto-generated Scripts to make running app.jar less verbose.
│   │   └── tmp -- Temporary Build Data
```

```
│   ├── build.gradle.kts -- Build Script - Defines Dependencies and Build Rules
│   └── src -- Source Code
│       ├── main -- The Main Application's Source Code
│       │   ├── kotlin
│       │   │   ├── App.kt -- Entry Point
│       │   │   └── cli -- Files used for generating the Command Line Interface
│       │   └── resources -- Currently Empty, contains resources required to be accessible
at runtime.
│       └── test -- Source Code for Unit Tests
├── archivetool.iml -- IDEA Project File
├── build -- Project Wide Build Cache
├── buildSrc -- Library used within other Subprojects' builds to prevent code duplication.
│   ├── build -- Build Cache
│   ├── build.gradle.kts -- Build Script for buildSrc
│   ├── settings.gradle.kts -- Settings File for buildSrc. Required as this is treated as a
fully-seperate Gradle Project.
│   └── src/main/kotlin
│       └── kotlin-jvm.gradle.kts -- Script providing definitions for other build files.
Used as a "plugin" elsewhere.
├── gradle -- Files used to help distribute a unified Gradle Version
│   ├── libs.versions.toml -- "Version Catalog", describes Gradle Dependencies, so that
consistent versions are used across the project.
│   └── wrapper -- Files used to download the right version of Gradle on systems without it
installed.
├── gradle.properties -- File that lets us set Gradle-wide Settings. Used to enable Caching.
├── gradlew -- Unix Gradle Wrapper
├── gradlew.bat -- Windows Gradle Wrapper
├── gui -- Currently Empty, Space for a Compose Multiplatform GUI or similar.
├── lib -- Frontend Independent Code
│   ├── build -- Build Cache
│   ├── build.gradle.kts -- Build Script - Defines Dependencies and Build Rules
│   └── src
│       ├── main
│       │   ├── kotlin -- Contains Kotlin Sources
│       │   │   ├── logging -- Code used to generate and emit logs.
│       │   │   ├── pythoninterop -- Code used to help interface with Python Code
│       │   │   ├── switchboard -- The master switchboard, and interaction definitions.
│       │   │   └── util -- General Utilities, Miscellaneous Code.
│       │   ├── python -- Contains Python Sources
│       │   └── resources -- Currently Empty, contains resources required to be accessible
at runtime.
│       └── test -- Contains Test Definitions
├── README.md -- ReadMe file. Talks about the project and how to use the source code
(Currently Auto-generated)
└── settings.gradle.kts -- Project Settings file. Defines where to look for dependencies and
subprojects.
```

Training Box-Segmentation Models

Current Pipeline

Last updated on 02/09/2026

The current training pipeline uses the newest model released by Ultralytics, YOLO-2026-Nano, fine-tuned in two stages. The first stage starts from Ultralytics' pre-trained weights and fine-tunes on the seg-package dataset for roughly 100 epochs, giving the model a general notion of box-like objects. The second stage further fine-tunes on the StoreBoxes dataset, aiming to bring the model closer to "archive-specific" box appearances rather than the more synthetic/warehouse style boxes in the package-segment dataset.

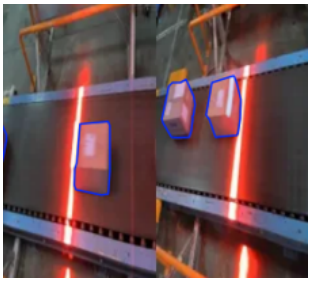
Initial Model Considerations

Remote models were considered to save on training overhead while still achieving the same results. However, in discussions with the client it was made clear that the hardware requirements are met on site and an in-house model may be preferable to avoid cost issues at scale.

Before deciding on the `YOLO-26n` model specifically, some other considerations included Sam-2 for its flexibility or a more directly algorithmic `OpenCV` approach using Box colours and dark boundaries to determine boxes. The issue for SAM-2 arises from the "dense" packing of the boxes, which is a known issue for these generic box-segmentation models. Open-CV was tested and may still be used alongside the YOLO model as a pre-processing step but it fails to generalise to differently coloured boxes, which have been shared as an significant sub-category of the dataset.

`seg-package` only training

The initial model, trained over ~2 T4 GPU hours courtesy of Google Colab, was trained using the Ultralytics provided package segmentation dataset. This was used as a baseline as the wide variety of lighting and environmental conditions help with initial fine-tuning for the model to approach generic "boxish" detection.



Labelled `package-seg` sample

Results from this model on the initial provided archive examples showed varying levels of success but consistently low confidence (<.2) and no fully identified images.



Low-confidence Prediction from the `package-seg` model



High-Confidence Prediction from the `package-seg` model

The issue seemed to stem from a general lack of data demonstrating boxes with only one face showing and a lack of discernible depth. To improve this multiple further fine-tuning datasets were considered, including retail-item detection, cardboard boxes and box cut-outs. Short tests on the `package-seg` trained model showed however that boxes in warehouses made for the best stand-ins for the archive boxes at the time being.

`StoreBoxes` Fine-tuning

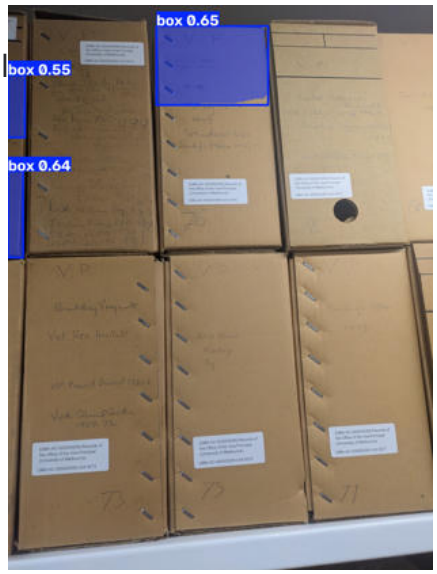
After training the 1st model on StoreBoxes for 40 Epochs (over ~8.5 Hours of GPU time), the segmentation showed promising improvement. Despite achieving lower confidences on its own validations, the model showed significantly more promise when identifying boxes in the client-provided examples.



Sample Training Batch from the `StoreBoxes` Dataset



These results already show increased improvement and generalisation to the `Archive` Dataset. While boundaries are still not ideal, the trend in confidence is indicative of increased performance and points towards further fine-tuning's effectiveness. Further fine-tuning would now preferably take place with our own recorded and labelled data; a site visit scheduled for `3 Sept 2026` should help with this collection but the improvement in model detection already suggests a final dataset of ~100-200 samples could be adequate for a baseline model that could integrate with user-provided validation.



Segmentation Output File Format

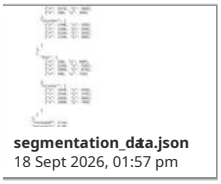
Segmentation output is to formatted using json file format.

Segmentation output is to be stored in a singular json file, even over multiple runs or sessions, using image filename as the key for the records.

Images are to be marked as “reviewed“ after segmentation review.

Vertexes are to be ordered clockwise, starting from the top-left vertex.

The json file below details the exact formatting the output should be in (2 image example):



segmentation_data.json
18 Sept 2026, 01:57 pm

Using Python Code from Kotlin

This article is about integrating a Python Module into Kotlin, for advice on *writing* easy-to-integrate Python Code, see Writing Well-Formed Python Code.

Introduction

The Kotlin Code treats Python Code in one of two forms:

1. A `PythonScript` that is a standalone file that does all of its behaviour when executed.
2. A `PythonModule` that is a series of files (or a folder) that has no once-off side effects on load. That is, on the top-level of all it's files, it only defines functions and variables.

Ideally, scripts are only used for testing, and mocking up ideas, and we use modules for everything else. This document only talks about `PythonModule`s.

The API for working with Python Code is internally quite complex, so that it is easy to work with externally, so it might be hard to grasp how to work with it by just reading the code.

Basic Usage

Preparation

This API is designed for the Python code to expect certain requirements, and for the Kotlin side to simply fulfill them, so implementation is easiest once you have functioning Python code, or at least an idea of what the Python code might need/generate. I strongly suggest at least reading the Python source before continuing, if you already have it.

Creating a Module

The `PythonModule` class has two constructors, and they function in a very similar way.

The only difference between these, is that the primary constructor wants a `Set` of file paths (relative to `<project>/src/<sourceSet>/python`), where the secondary constructor generates this set using a folder path (again, relative).

The class also takes a `Set` of `Injectable`s. `Injectable`s allow you to push Kotlin's functionality into global scope. Usually, the default (an empty set), will work fine, but check the Python code first.
There is a pre-defined `Injectable` called `Logger`, which creates a Python class called `Log`, which has the same API as the Kotlin `Log` object, deferring it's behavior to the Kotlin implementation.
If you want to create your own injectable, read Writing your own Injectable

> Performance Consideration
>
> When you construct a `PythonModule`, the files are immediately executed to populate the 'context', blocking the thread briefly. It may be

> worth using a thread or coroutine to defer the construction if this becomes problematic.

Reading the Python Environment

Once constructed, You can then access anything defined at the module's top-level.
The `PythonModule` class has the members `members`, and `methods`, the former being a set of every variable at the top-level, and the latter being filtered to only include callable methods. You can lookup a value in either of these using array indexation:

```
val module = PythonModule(...)
val method = module.methods["method_name"]!! // If the lookup fails, we get a null value.
```

Indexing these returns a "Polyglot `Value`", named as such because it is the internal type used by *Polyglot*, the library we are using to interpret the Python code.
`Value`s are quite flexible, they can be converted to and from many types automatically, and can be executed with `Value.execute(…arguments)`, where the arguments get automatically converted to `Value`s.

> Environment Stability
>
> One thing that you should consider as a developer is the state of the top-level. The library assumes that the top-level is semi-immutable, in that we expect anything defined at the top-level to exist for the module's entire lifetime, both not disappearing and not appearing as a side-effect of function calls, and you can make that assumption too. However, `Value`s mutate automatically as the value changes, so we cannot assume that top-level values have stayed the same.

Wrapping a `PythonModule`

Ideally, we don't want to expose `Value`s or other inter-operation oddities to the rest of our code. It is best when the impact of Polyglot is limited to one adapter class. Fortunately, there are a lot of tools at our disposal to make it easy to hide the truth from our consumers.

Safe Method Calls

One of the main reasons for writing a Python module at all is to have code harness the power of a Python library unavailable to Kotlin, like PyTorch or NumPy. Therefore, a lot of the time, we will simply be providing methods to the rest of the Kotlin package.

When wrapping a method, we need to:

- Find the internal method reference
- Convert arguments from JVM types to `Value`s
- Call the method
- Convert the return value from a `Value` to a JVM type.

Since `MemberSet`s (the type of `PythonModule.members` and `.methods`) have caching, you do not need to cache the method lookup yourself, many types also convert to `Value`s automatically. (Try using `PythonModule.context`'s `asValue`.) If a type doesn't have a converter by default, you can write a type-adapter as described in Wrapping Complex Types

Here is a complete example:

```
val module = PythonModule(...)
fun ComputeResult(input: Int, condition: Boolean): Float {
  val method = module.methods["compute_result"]!!
  // .asValue is automatically called on the arguments, but if you are using more complex types, you may want to use it to test that they convert as you expect.
  val result = method.execute(input, condition)
  // There are *loads* of methods both to convert to different types, but also to check if the type conforms properly, there will usually be one available.
  return result.asFloat()
}
```

When used like this, the `ComputeResult` method looks like a native Kotlin function to anything on the outside.

Safe Members

We can also use Kotlin's delegate functionality to help us create members that look like normal Kotlin members from the outside. There are three ways of doing this.

Private Backing Property

Another option is to use a private backing property for the member you want to shadow. This works well for simple values like primitives, especially if they are set with `val`. Doing more complex things with this is possible, but the cost might be quite high.

```
val module = PythonModule(...)
private val _member = module.members["member"]
val member
  get = _member.asFloat()
```

Additionally, `MemberSet` can be used as a property delegate if you are okay with explicitly being read-only, and sharing a name with the python structure.

```
val module = PythonModule(...)
private val member by module.members // will set this to module.members["member"]
```

When using a Custom Type Converter

`PythonType` comes with `jvmValue` and `polyglotValue` as members you can delegate to, and handles all logic, including changing the value.

```
val module = PythonModule(...)
val memberWrapper = PythonMember(module.members["member"]) // PythonMember extends PythonType
val member by memberWrapper.jvmValue
```

I intend to in the future create a concrete `PythonType` that is generic across a number of primitives.

Wrapping Complex Types

When a type is more complex, such as a class with complex behaviour or a type that Polyglot just has trouble converting to, you may need to write tools to help with manual conversion.

The process of doing so is actually surprisingly simple.

We provide two separate classes that together make it trivial to convert to and from types correctly, though they go hand in hand.

Converters

The first type is the `PythonTypeConverter`, which takes in a generic that matches your JVM type. You can think of this as a "strategy", that provides the simplest possible interface.

These are usually implemented as `object`s, since they don't typically require any state or a constructor, and they define two methods, one that converts a `Value` to a JVM type `T`, and one that does the conversion in the other direction.

A simple implementation for the `Boolean` primitive.

```
object BooleanConverter : PythonTypeConverter<Boolean> {
    override fun toJvmValue(polyglotValue: Value): Boolean {
        return polyglotValue.asBoolean()
    }

    override fun toPolyglotValue(context: Context, jvmValue: Boolean): Value {
        return context.asValue(jvmValue)
    }
}
```

While you can use this on it's own, it can at times be a bit cumbersome to work with, so it is best left to one-off operations, or for use in implementing a full type wrapper.

Type Wrappers

The other class that we can work with to make working with strange types easier, is `PythonType`. This loosely wraps the `PythonTypeConverter` interface. It takes a generic type, and a Polyglot context. In most cases, it works to use a setup with a private primary constructor and two secondary contructors, one for the JVM type, and one for `Value`.

Then, you can override `converter` with either a `PythonTypeConverter` instance, or an anonymous object, and the base class handles the rest.

Finally, using this class is easy. Just construct it, and delegate to either the `jvmValue` or `polyglotValue`. It automatically holds the inner value, converting it and updating it over all uses on `set()`.

Below is an annotated example.

```
// Private primary constructor that calls the base class.
class PythonBoolean private constructor(context:
WeakReference<Context>) :
    PythonType<Boolean>(context) {

    // Two secondary constructors that call the primary, then set one
of the members.
    constructor(context: WeakReference<Context>, inPolyglotValue:
Value) : this(context) {
        polyglotValue = inPolyglotValue
    }
    constructor(context: WeakReference<Context>, inJvmValue: Boolean)
: this(context) {
        jvmValue = inJvmValue
    }

    // You can use a PythonTypeConverter if you have one already
defined
    //override val converter = BooleanConverter

    // Or just use an anonymous object to implement the converter
without it's own class/object, if you aren't using it anywhere else.
    override val converter = object : PythonTypeConverter<Boolean> {
        override fun toJvmValue(polyglotValue: Value): Boolean {
            return polyglotValue.asBoolean()
        }

        override fun toPolyglotValue(context: Context, jvmValue:
Boolean): Value {
            return context.asValue(jvmValue)
        }
    }
}
```

⚠ There are some caveats with `PythonType`. This wrapper does not preserve the reference to the original value when mutating. This only matters when the value is actively being used in the context, rather than as a function parameter or similar.

Writing your own Injectable

Injectables are actually surprisingly lenient with what they actually do. Technically, you have full access to the Python context, and can do whatever you like. **However,** there is an expectation that your injectable doesn't abuse that privilege. The intended design is for an injectable to simply add one or more members to the context and leave.

Most injectables are just classes with values on them. You need to mark fields you want accessible with `@JvmField`, just since Kotlin's backing field system isn't handled well by Polyglot. Methods are best passed as `@JvmField val`s of lambdas implementing a Java functional interface type like `Consumer<T>`, `BiFunction<T,V,R>` or similar.

If you want to 'inject' a class, you may need to wrap it in lambda fields as is done in the `Logger` injectable.

It is also useful to implement a `toString` that better defines the class name so that is is more clear where problems are in logs.

Below is an example of an Injectable:

```
class MyInjectable : Injectable {
```

```
    override fun inject(ctx: Context) {
        ctx.getBindings("python").putMember("MyInjectable", this)
    }

    @JvmField val myMethod: Consumer<String> = { Log.log(LogLevel.Trace, "We are in a MyInjectable - $it") }
    @JvmField val myValue: Int = 42

    override fun toString(): String {
        return "Injectable.Logger"
    }
}
```

It might be worth providing a Python 'stub' to allow Python code to run without your injectable present.

Trying to read a variable and catching a `NameError` at the top-level is an easy way to do this.

The code on the right, for example, is a stub for the `Logger` injectable.

You may want to provide this to Python developers early so they can work locally with your injectables, without needing to run your code via Kotlin.

```
try:
    Log
except NameError:
    class Logger:
      def trace(self, msg):
        print("trace:", msg)
      def debug(self, msg):
        print("debug:", msg)
      def info(self, msg):
        print("info:", msg)
      def warn(self, msg):
        print("warn:", msg)
      def error(self, msg):
        print("error:", msg)
    Log = Logger()
```

External Dependencies

If the code has external dependencies, you can declare them as strings in a stye similar to `requirements.txt`, inside `lib/build.gradle.kts`, like so:

```
graalPy {
    packages = setOf("qrcode==7.4.2") // Python dependencies
}
```

Writing Well-Formed Python Code

Note: This document is currently notes from a conversation, and not in a good structure right now.

TLDR; With a few caveats, most Python code just works, but there are some things to consider to make it easier to integrate

Caveats

- no local imports, since it is concatenated at load time
- injectables will need a standalone wrapper

Tips

- you can check for the `KOTLIN` variable to know if you are running under the kotlin API or not. this can be used to specify behaviour if, for example, you need to import something (since kotlin will have it automatically imported already)

```python
# Example: Import `other_file.py` from the same directory.
# When in Kotlin, the file will already be in context,
#   and importing will cause a crash, because it can't find the file.
# But, if running under `python this_file.py`,
#     you need to manually import the other file or it won't be detected.
try:
  KOTLIN
except NameError:
  import "other_file"
```

- if it exists in kotlin, maybe ask about injecting it
- most types and classes just work, but you might run into something
- the files are loaded in pseudo-random order so it is best to avoid doing things
- there is no easy way to rerun the top-level so avoid any behavior there
- packages are defined in `lib/build.gradle.kts`, they look like lines in a `requirements.txt`. Most should work, but it is best to test that it does before over-committing to an implementation.
- things like `os` should just work when imported
- having some notion of "these are the types of these arguments and the type of what I return" is useful, because on the kotlin side, the code has to convert the weakly-typed python variables to strongly-typed kotlin variables.
  despite this, the variables can be of a 'vague' type like 'it returns an int if it fails or a dict if it succeeds' *will work*, but aren't ideal, since it may take more effort to make it concrete on the kotlin side.
- The API is designed around having the wrapper live on the Kotlin side, so don't stress too much about making arguments and return types "simple" or anything, a surprising amount just works.

Some Perspective

It might help to understand how your code fits within the project, and how the other side works.

I would suggest looking at Using Python Code from Kotlin | Basic Usage or at the function `load python module from fileset` in `lib/src/test/kotlin/PythonInteropTest.kt` for a bit of an idea of how things interact. The other tests use type wrappers and other behavior that is deceptively simple, and mostly used to make the kotlin code *feel* natural, and doesn't really impact your code.

🐘 Using Gradle with this Project

Gradle is a very powerful dependency management and build tool, that we are using for this project. This is a very brief guide to using the Gradle tooling, as it is setup for the project, and not a guide to configuring Gradle.

If you are looking to change build settings, they are all defined in Kotlin scripts, and you can find where they are here Project Structure | File Structure

Wrapper

This project uses a Gradle wrapper. Gradle generates these to allow us to consistently use the same Gradle version across systems. Rather than executing things with your system Gradle, call either `./gradlew.bat` on Windows, or `./gradlew` on any other platform. The rest of this document presumes that you are prefixing each command with `./gradlew`, for example, when referring to `app:run`, you would need to execute `./gradlew app:run`.

Tasks

Tasks are units of code-building that Gradle uses to isolate certain parts of the process.
Most of these are internal to Gradle, and not particularly useful to us; but here are a selection of useful ones.

- `build` will build a project and its dependencies.
- `run` will execute the 'main' function of a project. You can pass command line arguments with the flag `--args=""`.
  (You likely want to use `app:run` as described 🐘 Using Gradle with this Project | Subprojects)
- `test` will run all tests defined in a project.
- `ktfmtFormat` is added by an extension, and will auto-format all of the Kotlin code in our project.
- `check` will run `test`, and then additionally verify that all files are compliant with the autoformatter.

Gradle also allows for you to 'chain' these tasks in a single invocation. For example, `./gradlew ktfmtFormat test` will first run the autoformatter, then run every test.

Subprojects

The project currently defines three subprojects, `app`, `gui` and `lib`.

`app` contains code defining the Command Line Interface for the application.

`gui` is a holdover from when we were considering a full GUI for our tool.

`lib` defines the business logic.

If you want to perform an action on a specific subproject, you can address it with `<subproject>:<action>`, for example, `lib:build` will build only the `lib` project. This is particularly useful with `run`, where without specifying a subproject, both `app` and the `gui` stub will run.

JavaCPP

By default, JavaCV (the library we use to pull the image processing library OpenCV) will download shared libraries for OpenCV for *every* platform. This is usually not necessary. You can pass `-Djavacpp.platform=<platform>`. Platforms are in the format `<os>-<arch>`, for example: `linux-x86_64`.

Process View

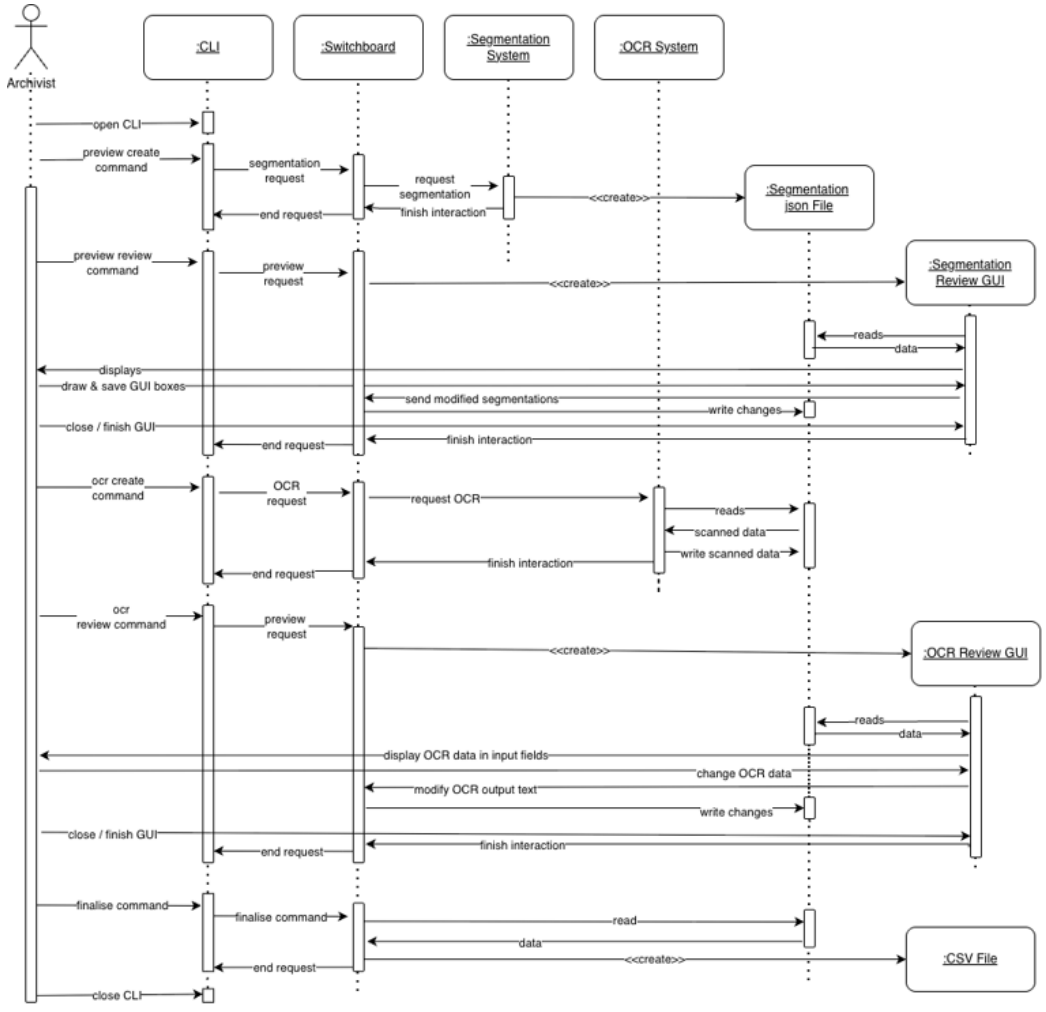
The software process comprises of the user calling four primary commands in order: preview create, preview review, finalise, and finalise review. These comprise the intended pipeline to yield the final formatted output file. These four commands are aggregated under a singular “run“ command which the user may run.

The preview review and finalise review commands are optional in yielding the final output.

All of the commands that the user will call are described in the documer CLI Design and Behavio ur

Standard Use Case

This diagram demonstrates the interactions of the system when the user runs the commands for the four main components of the software in intended order.



Changelog:

4/10/26

- Extended to include OCR review stage

8/10/26

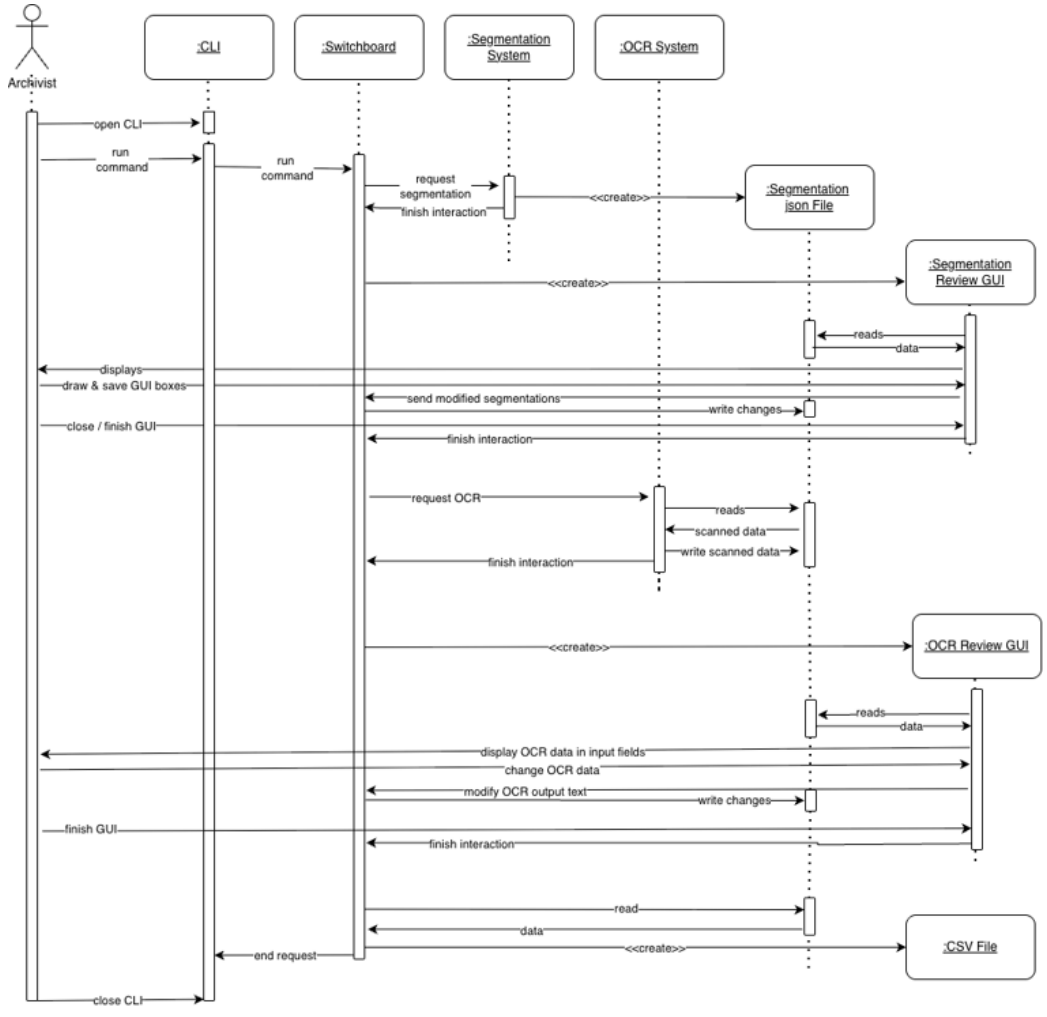
- Defined return requests for completing switchboard and CLI commands to make command sequencing more defined
- Added opening and closing CLI to the sequence

15/10/26

- Changed “finalise create“ and “finalise review“ command names to “ocr create” and “ocr review”
- Clearly defined export with “finalise” command and CSV output

“run” Command Use Case

The diagram below shows the system sequence of the run command.



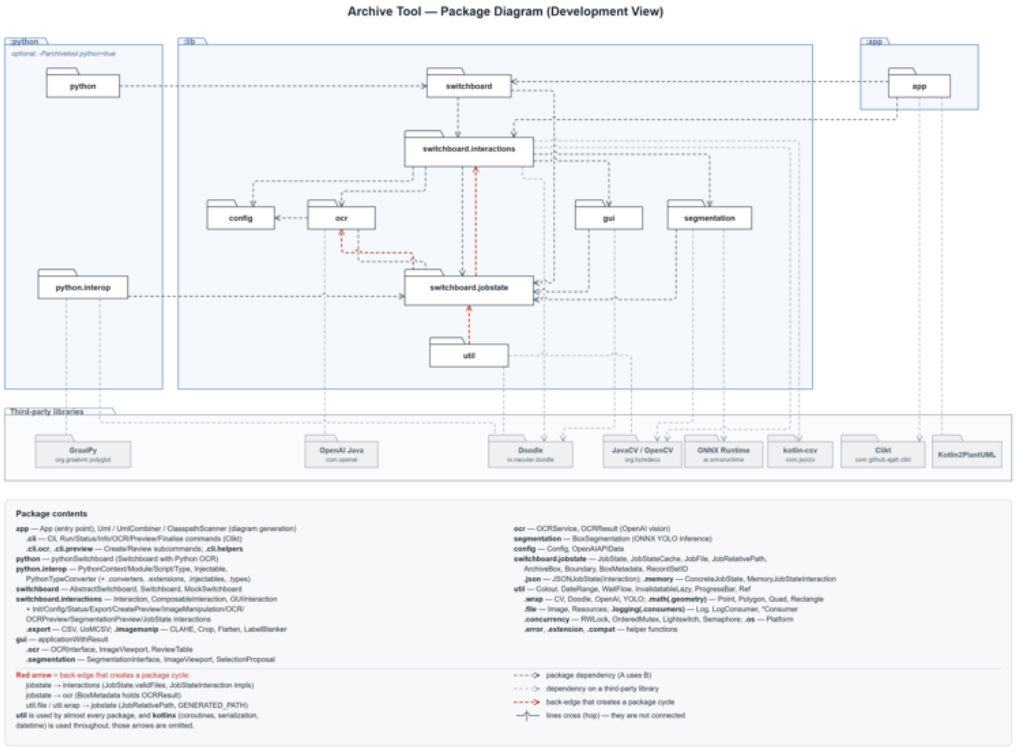
Changelog:

15/10/26

- Clearly defined export stage with CSV output

Development View

Package Diagram



Architectural Goals

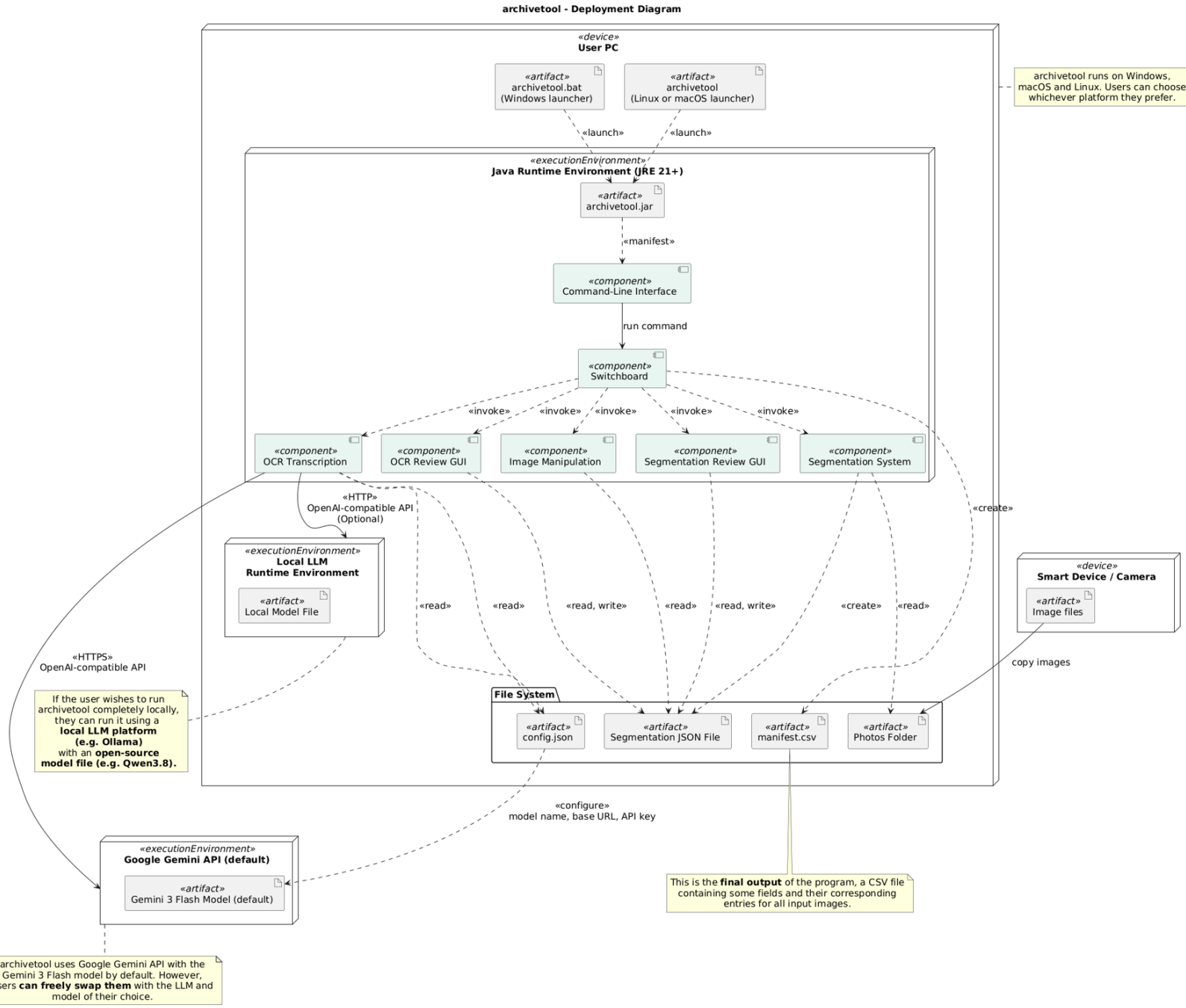
Requirement	What	Why	How
Scalability	Handle a large set of photos in one run without running out of memory or crashing	Thousands of boxes need to be covered across the whole archive	• Work tracked per file in switchboard.jobstate (JobFile, JobStateCache) • JSON job state written incrementally
Efficiency	Process images as quickly as possible and avoid wasted work	Large volume of boxes; processing time and API cost add up	• Used concurrency primitives in util.concurrency to enable parallel work • Skip OCR on boxes without an ID sticker
Reliability / unattended runs	A batch can run unattended and recover	Archivists shouldn't have to watch the tool;	• CLI entry point (app) runs the whole pipeline

	if something fails part-way	a crash shouldn't lose work	• Progress saved in switchboard.jobstate so a failed run can resume • Error helpers in util.error
Accuracy	Boxes are segmented correctly and text and ID labels are read correctly	Wrong transcriptions or missed boxes make the inventory unusable	• YOLO segmentation model (segmentation) • ID label separated from the rest of the box (LabelBlanker) • Image pre-processing: CLAHE, Crop, Flatten (interactions.imagemanip) • OpenAI vision OCR (ocr)
Consistency	Output fields are generated the same way for every box, without excess categories	Archivists sort and compare records across a collection	• Fixed metadata model (BoxMetadata, DateRange) • Fixed CSV schema (CSV, UoMCSV)
Traceability	Each output record links to the archive's existing reference number and its source image	Records must be locatable on the shelf; image position alone means nothing	• Records keyed by RecordSetID (existing UMA reference) • JobFile / JobRelativePath link each record to its source image
Usability / user control	Archivists can see, adjust and correct what the system produced	Users need to feel in control of, and confident in, automated results	• Segmentation review GUI (gui.segmentation): adjust box outlines

			• OCR review GUI (gui.ocr): correct transcribed text
Systematic review	Reviewers can prioritise the least reliable results and review work in batches	Reviewing every box equally is too slow	• Review table in gui.ocr • Flag low-confidence results • Merge / deduplicate results from multiple runs
Flexibility / modifiability	Components can be adjusted, swapped or reused on their own	The ML model may need tuning; parts of the pipeline may be reused elsewhere	• Interaction interfaces composed by Switchboard • Optional :python module (GraalPy) so ML parts can be adjusted in Python • Mock* implementations for testing
Observability	The application produces clear logs	System admins need to spot when something goes wrong in an unattended run	• util.logging with file, stdout and stderr consumers
End-to-end integration	One cohesive application covers the whole workflow	Users shouldn't have to install or connect separate tools	• Single CLI entry point (app) • Switchboard orchestrates every step from input to export
Interoperability of output	Output is in a well-structured format that UMA and data scientists can import	The inventory has to be usable outside the tool	• CSV export (interactions.export, kotlin-csv)
Credential confidentiality	The OpenAI API key isn't exposed	A leaked key costs money and is a security	• Stored in local config (config.OpenAIAPID

		risk	ata)

Physical View



Link to the deployment diagram on PlantUML:

Changelog:

21/10/26

- Include **CLI** and **Switchboard** components in the diagram.
- Add link to the deployment diagram on PlantUML.
- Rename “Box Segmentation” component as “Segmentation System”
- Remove “CSV Export” component from the diagram (merged into Switchboard).
- Add notes to provide useful clarification.

High Fidelity Prototype - Segmentation Review

Annotations refer to 🚀 Component & Interaction Specification + Example Workflow - Segmentation Review

Initial Window Dimensions: 1100px x 1000px



Changelog:

21/09/26:

- Made GUI - 02 colour fill a lighter blue to make outline more visually distinct

24/09/26:

- Added GUI - 10 for automated pipelining and give the user a clear way to end the process

🚀 Component & Interaction Specification + Example Workflow - Segmentation Review

Component & Interaction Specification

ID	Component	Function & Behaviour	Input	Output	Notes
GUI - 01	Image Preview	Allows user to see current image with segmentation pair (both boxes and stickers) drawn on	All images in target directory	Visual representation	Displayed in a 4:3 aspect ratio Each box outline is a randomised saturated colour Label outlines are always blue
GUI - 02	Segmentation Selection	Highlights each vertex of segmentation pair When a selection is made, user can drag vertexes to define a new segmentation area User can drag the area itself to move the entire segmentation	User clicking on edge or within segmentation area User dragging vertex	Visible selection of segmentation with vertexes and blue highlight	Only one segmentation pair can be selected at once

		User can drag the area of just the label to move just the label, clamped to the box area	User draggi ng area		
GUI - 03	Previous Button	Changes image preview to previous image file in batch	User click	New image preview	Not clickabl e when first image in batch
GUI - 04	Next Button	Changes image preview to next image file in batch	User click	New image preview	Not clickabl e when last image in batch
GUI - 05	Confirm Button	Confirms changes made in segmentation selection	User click	Saves the new mapping to associat ed JSON segment ation file	Only clickabl e when selectio n is active Remov es selectio n when clicked

GUI - 06	Create Button	Allows user to create a segmentation pair in the centre of the GUI	User click	Adds new mapping to associated JSON segmentation file	Created button is automatically selected
GUI - 07	Delete Button	Allows user to delete the selected bounding box, removing its selection too	User click Existing selection	Removes mapping associated with JSON segmentation file	Only clickable when selection is active
GUI - 08	File Counter	Counts the file number of the displayed image against the total amount of images in the target directory	Image count in target directory	Text displaying current and total image counts	Sorted by accuracy metric ascending
GUI - 09	Image Scaler	Allows user to zoom in and out of preview image	User click	Zooms in on preview image	Auto scales image when loaded to fit viewport Selection vertex size stays consist

					ent regardless of zoom
GUI - 09a	Zoom In (+)	Zooms in on image preview	User click	Zooms in on image preview	N/A
GUI - 09b	Zoom Out (-)	Zooms out on image preview	User click	Zooms out on image	N/A
GUI - 09c	Fit (Fit)	“Fit” centres images in 4:3 ratio of viewport and adjusts zoom so image is fully visible on preview	User click	Scales image to fit viewport	N/A
GUI - 10	Finish Button	Allows the user to close the window	User click	Closes the window	N/A

Example Expected Workflow

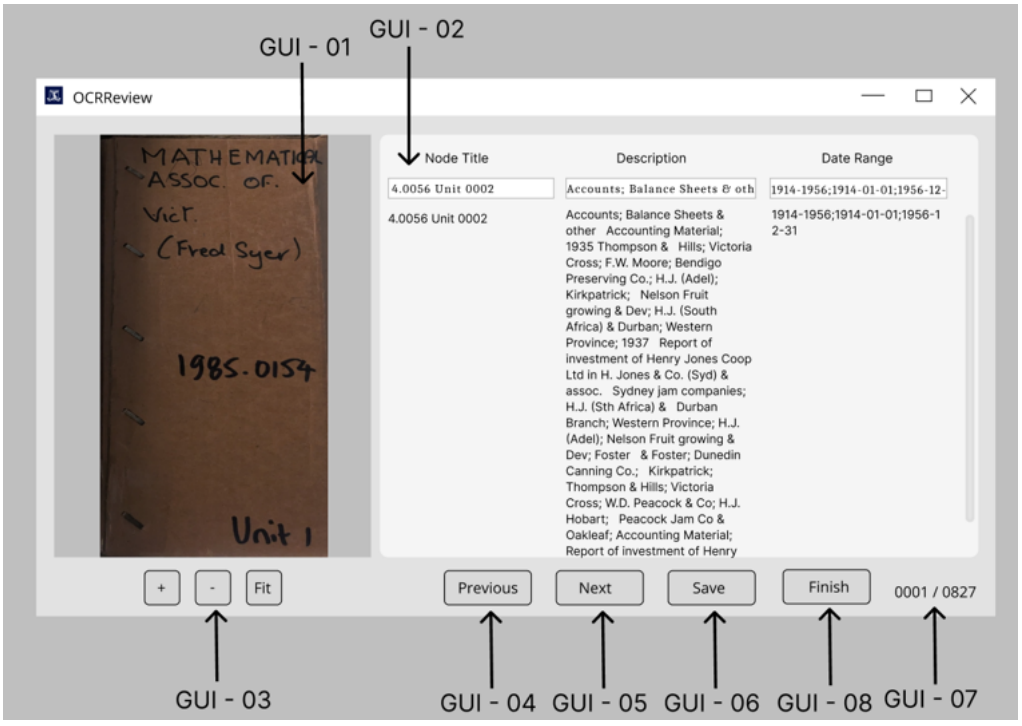
In this scenario, the user wants to manually adjust an existing segmentation, confirm, and then proceed to the next image.

1. GUI - 01 loads and user clicks GUI - 09c
2. GUI - 09 zoom display updates
3. User creates a selection with GUI - 02
4. User drags multiple selection vertexes to new positions
5. User clicks GUI - 05
6. GUI updates associated image segmentation JSON file
7. User clicks GUI - 04
8. GUI - 08 increments by one
9. The next GUI - 01 is loaded with its associated segmentation file

High Fidelity Prototype - OCR Review

Annotations refer to Component & Interaction Specification + Example Workflow - OCR Review

Window Dimensions: 1200px x 675px (16:9)



Changelog:

4/10/26:

- Changed initial window dimensions from 1600px x 900px to 1200px x 675px as it was practically too big for most standard displays to fit fully (still 16:9)
- Added GUI - 08 for automated pipelining and give the user a clear way to end the process
- Centred buttons beneath respective GUI elements as it gives more visual balance to the window and makes clear what the buttons correspond to
- Changed GUI - 02 to separate input field and output preview due to limitations with the Doodle Kotlin UI Framework

Component & Interaction Specification + Example Workflow - OCR Review

Component & Interaction Specification

ID	Component	Function & Behaviour	Input	Output	Notes
GUI - 01	Image Preview	Allows user to see a flattened image of the current box	All box images of all images in target directory	Visual representation	Displayed in a 3:4 aspect ratio
GUI - 02	OCR Review Table	Allows user to type to change the output text for the 3 fields the OCR text extraction outputted for that box, with a visual display showing that input over multiple lines Output file for given box is already formatted to text in the text field cells	OCR output file User clicking on text User dragging text User keyboard	User inputted text in the text field Visual display of what the user has typed in the text field	Column headers cannot be modified If any cell goes out of the bounds of the table viewport, user can use the scrollbar on the right side of the table to

					scroll up/down the entire view
GUI - 03	Image Scaler	Allows user to zoom in and out of preview image	User click	Zooms in on preview image	Auto scales image when loaded to fit viewport Selection vertex size stays consistent regardless of zoom
GUI - 03a	Zoom In (+)	Zooms in on image preview	User click	Zooms in on image preview	N/A
GUI - 03b	Zoom Out (-)	Zooms out on image preview	User click	Zooms out on image	N/A
GUI - 03c	Fit (Fit)	“Fit” centres images in 3:4 ratio of viewport and adjusts zoom so image is fully visible on preview	User click	Scales image to fit viewport	N/A

GUI - 04	Previous Button	Changes image preview to previous box image file in batch	User click	New image preview	Not clickable when first image in batch
GUI - 05	Next Button	Changes image preview to next box image file in batch	User click	New image preview	Not clickable when last image in batch
GUI - 06	Save Button	Saves the current state of the table	User click	Updated text	N/A
GUI - 07	Box Counter	Counts the number of the displayed box against the total amount of boxes of all images in the target directory	Box count of all images in target directory	Text displaying current and total box counts	N/A
GUI - 10	Finish Button	Allows the user to close the window	User click	Closes the window	N/A

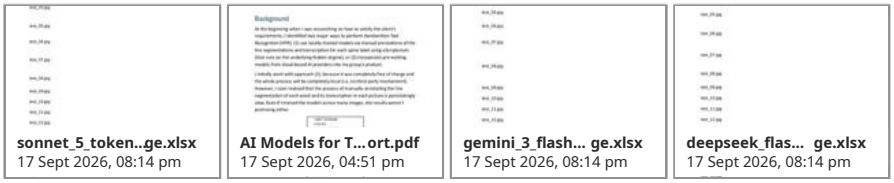
Example Expected Workflow

In this scenario, the user wants to manually adjust the description of an OCR output file for a box, save, and then proceed to the next box image.

1. Window loads and GUI - 03 zoom display updates
2. User creates a selection of text in the “Description” cell of GUI - 02, deletes the text, and types in some new text
3. User clicks GUI - 06
4. GUI updates associated OCR file/state for box

5. User clicks GUI - 05
6. GUI - 07 increments by one
7. GUI-01 and GUI-02 are loaded using the next boxes associated OCR output file

AI Models for Text Recognition: Evaluation Report



Background

At the beginning when I was researching on how to satisfy the client's requirements, I identified two major ways to perform Handwritten Text Recognition (HTR): (1) use locally-trained models via manual annotations of the line segmentations and transcription for each spine label using eScriptorium (that runs on the underlying Kraken engine); or (2) incorporate pre-existing models from cloud-based AI providers into my group's product.

I initially went with approach (1), because it was completely free of charge and the whole process will be completely local (i.e. no third-party involvement). However, I soon realised that the process of manually annotating the line segmentation of each word and its transcription in each picture is painstakingly slow. Even if I trained the models across many images, the results weren't promising either.



Figure 1 - Transcription of test_27.jpg using the locally-trained models in approach (1). The models has been trained over 30 images, but there still exists serious confusions between the characters, expecially in the canonical number and keys.

As such, after conveying my finding to the client and knowing that he has previously attempted to use AI models to perform HTR, I proceeded to evaluate the feasibility of approach (2). For this evaluation, I have chosen Google's *Gemini 3 Flash Preview*, DeepSeek's *V4.1 Flash*, and Anthropic's *Claude Sonnet 5* as my test subjects.

Methodology

To ensure fairness, each AI model will be fed with the same 26 images (of distinct, individual spine labels and be given the same prompt:

> *“Extract all text from this image exactly as it appears. Preserve line breaks and formatting. Do not summarize or add commentary. Ignore strikethroughs and random numbers in the corner."*

The results were written into a CSV file, which I then converted it to Excel for better formatting. Each row of the Excel file contains the file name of the image, description, input token cost, output token cost, and total token cost (the sum of input and output).

Since these APIs require credits, from which they will deduct the cost (based on the number of tokens used) from the credit balance, I evaluated each AI model based on its input and output token costs and the corresponding price. On top of that, I will make some observations each model’s ability to accurately extract the texts, but not so much, since they all performed similarly with a few variations. The client is more than encouraged to look at the text transcriptions for each model in the Excel files to observe the performance, so **I will leave it up to the client to give the verdict on which model is the best while considering the costs.**

It can be said that the main objective of this evaluation is to find the key balance between performance and efficiency; identify a model that performs well, but at the same time does not require too many tokens.

Models

Gemini 3 Flash Review

Pricing

While Gemini API provides a Free tier for developers with small projects, it unfortunately comes with a few caveats. Firstly, Free users are only allowed to make at most 5 requests per minute (RPM) and up to 20 requests per day (RPD), or in this context, they can only extract texts from 5 images per minute and 20 images for the whole day, or a total of only 140 individual spine labels per week. Furthermore, Gemini explicitly states that they will use Free users’ content (i.e. prompts and responses) to improve their products. Therefore, if the client prioritises high efficiency and guarantee to strict privacy, then **Gemini API’s Free tier is unfeasible**.

For Gemini API’s Paid tier, the standard pricing for the Gemini 3 Flash Preview model is as follows:

- Input price: **0.50 USD per one million tokens**
- Output price: **3.00 USD per one million tokens**

Looking at the results from *gemini_3_flash_token_usage.xlsx*, the average input token cost is **1122.96,** and the average output token cost is **77.69**. Using the above pricing, processing

1,000 images will cost about **0.74 USD in total.**

Performance

The extracted texts seem to be coherent for all images with minimal noise, with the exception of *test_08.jpg*, which may or may not be caused by a bug in the code.

DeepSeek V4.1 Flash

Pricing

DeepSeek's pricing has two sets of rates: **peak rates** (01:00 – 04:00 UTC and 06:00 – 10:00 UTC, Monday through Friday) and **off-peak rates**, which are half of the peak rates. The pricing per one million tokens is as follows:

- Peak rates
  - Input price: **0.30 USD**
  - Output price: **1.20 USD**
- Off-peak rates
  - Input price: **0.15 USD**
  - Output price: **0.60 USD**

Looking at the results from *deepseek_flash_token_usage.xlsx*, the average input token cost is **484.77,** and the average output token cost is **236.23**. Using the above pricing, processing 1,000 images will cost about **0.43 USD during peak hours** and **0.21 USD during off-peak hours**.

Performance

The text extraction seems to have functioned properly with near-perfection transcription besides a few odd characters, as shown in *test_05.jpg* and *test_26.jpg*.

Claude Sonnet 5

Pricing

The pricing for Sonnet 5 API is as follows:

- Input price: **2.00 USD per one million tokens**
- Output price: **10.00 USD per one million tokens**

Looking at the results from *sonnet_5_flash_token_usage.xlsx*, the average input token cost is **908.31,** and the average output token cost is **108.35**. Using the above pricing, processing 1,000 images will cost about **2.90 USD in total.**

Comparison

(Prices are in USD per one million tokens)

Cost/Model	DeepSeek V4.1 Flash	Gemini 3 Flash	Claude Sonnet 5
Input cost	0.30 (peak), 0.15 (off-peak)	0.50	2.00
Output cost	1.20 (peak), 0.60 (off-peak)	3.00	10.00
Total cost (for 1,000 images)	0.43 (peak), 0.21 (off-peak)	0.74	2.90

Deployment

This project is built and “deployed” as artifacts by Github Actions.

It uses the Gradle Wrapper, and the built-in distribution plugin to generate . However, the version deployed on Github Actions is a “fat” version that is multiplatform and slightly larger and slower than if it were built for a single system. Therefore, while we have the option to deploy the code very easily, at this stage we plan to simply recommend building the software on and for the target machine, since build times are relatively quick and instructions very simple.
Additionally, since the code runs primarily locally and doesn’t have many “tricks” required to get the most out of the software, with no servers or anything of the sort required, the typical advantages of deployment aren’t as important here.

Changelog

October 23rd - *Amy* - Initial description of existing solution.

Progress Overview

Below is a link to relevant details for each progress report item.

Tools

Tools are linked where appropriate, an overview exists Project Tools.

Process

Team Structure

Team Structure is documented in Team Allocations . In particular Role Descriptions gives a formal overview of each role.

Agile Process

Our Agile User Stories are documented at User Stories.

Sprint Plan and Sprint Retrospective documents are found a Sprints .

Daily standup is documented at the start of each team meeting document. Found Team Meeting s .

An up-to-date overview of our progress can be found on Trello a T8-2 UoM Archives .

Meetings

We publish the agenda for each meeting in advance on our main Slack channel (
and record
minutes for each, available in Team Meetings.

Internal Communication

We communicate using Slack.

Decision Making Process

We have an outline of the formal decision making process outlined within c Team Contract.

Communication with Client

Along with a Meeting on Monday, October 5th, we also have regular communication via. email. The key threads are reproduced in Client Emails.

Artifacts

Requirements

These are outlined in Project Requirements . Most of them are based on communication with the client, and the decision-making surrounding them is primarily discussed in the minutes Team M eetings.

Frontend Design

Our project comprises of 3 separate user interfaces, each of which are separately documented.

Our outline of our CLI is at CLI Design and Behaviour.

The High Fidelity design for the Segmentation Review GUI is at High Fidelity Prototype - Segmentation Review, with its component descriptions at Component & Interaction Specification + Example Workflow - Segmentation Review.

The High Fidelity design for the OCR Review GUI is at High Fidelity Prototype - OCR Review , with its component descriptions at Component & Interaction Specification + Example Workflow - OCR Review.

User-facing documentation is located in the repository docs, under the docs/ folder and in README.md. These links will obviously need to be updated after hand-over.

Architectural Design

There are several documents describing parts of our architecture in System Design & Architecture .

The approach to our project's architectural design since the last progress report has changed significantly. This mainly comes from centring it around architectural views, which has helped significantly in getting the entire team on the same page in understanding the switchboard structure, and how each part of the project relates. This has been an especially important change because the last 2 sprints have focused heavily on integrating each of our initially separate parts together. Found in 4+1 Architecture .

The report for our out evaluation of different approaches for the text extraction process is found at AI Models for Text Recognition: Evaluation Report

While our architecture is quite complex on the surface, it is quite intuitive and very configurable once you understand the `Switchboard` dependency injection setup.

Coding

Repository

Our repository has detailed guidelines on commit style and PR review, outlined in Version Control Standards.

The repository will be handed over more-or-less as-is to the client, who plans to publish it as open source.

Code Style

We also have rules about code style, defined here Style Coding Standard.

Code style for Kotlin is tested for automatically using GitHub Actions.

Testing

Our Test Plan is found here Test Plan , and our Test Report Document is located at Test Report. More integration testing specifically will be done before handover.

Deployment

Our minimal deployment documentation is located Deployment. Additionally, there is documentation of how to compile and how to configure the build on our Github repository, in `README.md`.

Others

Maintenance

While there is not a very concrete metric for maintenance, the analytics of the confluence workspace are available at

Response to Feedback

The main action our team has made in response to feedback has been the structuring of our architectural design around the 4+1 model.

Another was altering our Do/Be/Feel/Who, Motivational Model, and User Stories after our feedback from Leon Sterling. This mainly focused on our requirements being explained in an overly complicated way.